

# Introduction to NIBRS and SQL

Greg Ridgeway

2026-07-13

## Table of contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>National Incident-Based Reporting System</b>   | <b>2</b>  |
| <b>2</b> | <b>Structured Query Language (SQL) and SQLite</b> | <b>3</b>  |
| <b>3</b> | <b>Acquiring the data</b>                         | <b>4</b>  |
| <b>4</b> | <b>The NIBRS format lookup table</b>              | <b>6</b>  |
| <b>5</b> | <b>Setting up the SQLite database</b>             | <b>10</b> |
| <b>6</b> | <b>Loading the data into SQLite</b>               | <b>10</b> |
| <b>7</b> | <b>Exploring NIBRS with SQL</b>                   | <b>13</b> |
| 7.1      | SELECT, FROM, and WHERE . . . . .                 | 13        |
| 7.2      | GROUP BY and aggregate functions . . . . .        | 16        |
| 7.3      | ORDER BY and DISTINCT . . . . .                   | 18        |
| 7.4      | Joining tables . . . . .                          | 26        |
| 7.4.1    | Loading lookup tables into SQLite . . . . .       | 26        |
| 7.4.2    | INNER JOIN . . . . .                              | 27        |
| 7.4.3    | LEFT JOIN . . . . .                               | 32        |
| 7.5      | Date functions . . . . .                          | 36        |
| 7.6      | Window functions . . . . .                        | 45        |
| 7.7      | Indexes . . . . .                                 | 54        |
| <b>8</b> | <b>Exercises</b>                                  | <b>56</b> |
| <b>9</b> | <b>Answers to the exercises</b>                   | <b>57</b> |

# 1 National Incident-Based Reporting System

The FBI established the Uniform Crime Reporting (UCR) Program in 1930 to generate reliable crime statistics for law enforcement administration, operation, and management. It has been the primary source of crime data in the United States for decades.

The UCR's Summary Reporting System (SRS) is the best-known component of the UCR. About 18,000 law enforcement agencies, including municipal police departments, sheriff's departments, campus police, transit police, park police, and many other agencies, reported monthly counts of Part I crimes (murder and non-negligent manslaughter, forcible rape, robbery, aggravated assault, burglary, larceny theft, motor vehicle theft, and arson), often called "index crimes." The SRS also tracked monthly counts of Part II crimes, such as simple assault, fraud, vandalism, drug offenses, and driving under the influence. Even though reporting to the SRS was voluntary, almost all law enforcement agencies reported their data, offering fairly comprehensive coverage of crimes reported to the police in the United States.

While the UCR SRS has provided valuable crime data for many years, it has several notable limitations. The SRS focused on a limited number of crimes, potentially overlooking important details and emerging trends. With SRS data researchers cannot separate out trends in shootings, identity theft, and cybercrimes. Shootings, for example, are lumped together in an aggravated assault category that includes a range of serious assaults like striking someone with a beer bottle. The SRS also operated under the "Hierarchy Rule," reporting only the most severe offense in a multi-offense incident. Lastly, the SRS only collected aggregate counts, meaning that detailed information about the context of the crimes, such as the characteristics of the victims and offenders, the extent of property loss or damage, the time, place, and context of the crime, and the relationships between these crime features, is lost.

The FBI introduced the National Incident-Based Reporting System (NIBRS) in the 1980s. NIBRS aimed to address the shortcomings of the SRS by capturing incident-level data and a comprehensive description of what happened in each incident. Unlike the SRS, NIBRS collects data on each individual crime incident, capturing detailed information about the offenses, victims, offenders, property, and arrestees. NIBRS records data on 52 "Group A" offenses and 10 "Group B" offenses, covering a broader spectrum of criminal activity. It did away with the Hierarchy Rule and collects data on all offenses within a single incident, providing a fuller picture of criminal activity. Because NIBRS is incident-based, we have full access to the multivariate relationships between features of crime incidents.

On January 1, 2021, the FBI officially retired the SRS marking a significant transition towards the exclusive use of the National Incident-Based Reporting System for crime data collection and reporting. Although the transition has been planned for almost a decade, many law enforcement agencies are yet to transition their information systems to report to NIBRS. As of this writing, four of the nation's largest states, California, New York, Pennsylvania, and Florida, essentially do not participate in NIBRS. You can find a [map describing NIBRS coverage](#) at the Bureau of Justice Statistics. The largest law enforcement agencies in those states regularly post crime

data to their local open data portals, but you will not find those data in NIBRS yet. The transition to NIBRS also makes the study of long-term national crime trends challenging. Any study that spans the SRS-NIBRS transition will have to grapple with inconsistencies and gaps in data as agencies adapt to the new system.

The FBI accepts data from law enforcement agencies through March of the following year. That is, the FBI accepted NIBRS data for 2023 through March 2024. Some crimes committed in December 2023, for example, may be solved or result in an arrest in April 2024. Such a case would not get marked as “cleared” in the NIBRS data since the clearance came after the NIBRS 2023 submission end date. Crimes in January 2023 may seem to be solved at a higher rate than crimes in December 2023 because of the March 2024 censoring. This is a feature of the data that simply requires care. As our earlier work with the NCVS emphasized, NIBRS also contains only crimes reported to the police, and reporting rates can vary greatly by type of crime.

## 2 Structured Query Language (SQL) and SQLite

The NIBRS 2023 master file contains over 73 million rows of data, far too large to load into R’s memory all at once, and too complex to manage as a collection of flat files. This is where a relational database management system becomes essential. Rather than storing all data in one monolithic file, a relational database organizes information into multiple tables that are linked together by shared keys. In the case of NIBRS, that means separate tables for offenses, victims, offenders, property, and arrests, all linked by the ORI and incident number that identify each incident. This structure eliminates redundancy, keeps related data together, and makes it possible to answer complex multi-table questions efficiently. Perhaps most importantly for our purposes, a relational database lets you query a specific slice of the data without loading everything into memory first. A question like “how many completed burglaries occurred in Texas in 2023?” can be answered in a fraction of a second against 73 million rows without R ever seeing any row outside of Texas.

Structured Query Language, or SQL (pronounced “sequel”), is the standard language for communicating with a relational database. SQL has been standardized by the American National Standards Institute since 1986 and is supported by virtually every database system in existence. One of SQL’s most important characteristics is that it is a *declarative* language rather than an *imperative* one. In R you write code that tells the computer *how* to do something: loop over these rows, filter this column, group by that variable. SQL instead lets you describe *what* you want, and the database engine figures out the most efficient way to retrieve it. You write `SELECT crime_type, COUNT(*) FROM offense GROUP BY crime_type` and the engine decides on its own how to scan, filter, and aggregate the data as fast as possible. This frees you to think about the question rather than the mechanics of retrieval, and it means the same SQL query will run correctly (and fast) whether the table has a thousand rows or a billion.

We will use [SQLite](#) as our database engine. SQLite differs from most database systems in one important way: it stores the entire database, all tables, indexes, and metadata, in a single ordinary file on disk. There is no server to install, no background process to manage, and no administrator credentials to configure. You simply point your code at the `.db` file and start querying. SQLite is also released into the public domain, meaning there are no licensing fees, no restrictions on use, and no vendor lock-in. Despite its simplicity, SQLite is far from a toy system. It is the most widely deployed database engine in the world by a large margin. Estimates place the number of active SQLite databases in the trillions. It runs inside every iPhone and Android phone, every Mac and Windows 10 or later machine, every Chrome and Firefox browser, every Airbus A350 flight information system, and countless other devices and applications. The skills you develop querying SQLite transfer directly to PostgreSQL, MySQL, Microsoft SQL Server, Oracle, and other enterprise database systems, since the core SQL language is largely the same across all of them.

### 3 Acquiring the data

The complete NIBRS data are available from the FBI's Crime Data Explorer [downloads page](#) under the Master File Downloads section heading. From the dropdown menu select "National Incident-Based Reporting System (NIBRS)". The compressed data file, `nibrs-2023.zip`, is over 500 Mb so downloading will take some time. It has over 73 million rows of data. Unzip `nibrs-2023.zip`. It is very big... about 6 Gb. I then run `gzip 2023_NIBRS_NATIONAL_MASTER_FILE.TXT` from the Terminal pane in RStudio (not the Console pane, the Terminal pane is typically one tab to the right of the Console pane).

NIBRS is too large to work with comfortably in R alone, so we will load it into a SQLite database. Once the data is in SQLite, we can query any subset of the 73 million records nearly instantaneously without ever loading the entire dataset into R's memory. We will use the `RSQLite` package to connect R to SQLite.

Let's start by loading the libraries we will need and peeking at a few lines of the data.

```
library(readr)
library(dplyr)
library(RSQLite)

# Set rebuildDB to TRUE to rebuild the database from the master file
#   (deletes any existing nibrs2023.db and re-reads all 73 million rows,
#   which takes several minutes). Set to FALSE to reuse the existing
#   nibrs2023.db and skip the slow rebuild.
rebuildDB <- FALSE
```

```
scan("NIBRS/2023_NIBRS_NATIONAL_MASTER_FILE.txt.gz",
      nlines=5, what="", sep="\n")
```

|     |                             |                        |          |
|-----|-----------------------------|------------------------|----------|
| [1] | "BH50AK00101000000000000000 | ANCHORAGE              | AK1C941Y |
| [2] | "BH50AK00102000000000000000 | 20210101FAIRBANKS      | AK4 941Y |
| [3] | "0150AK0010200230000003     | 20230101 11010010100 N |          |
| [4] | "0250AK0010200230000003     | 2023010123DCN 04       | 88"      |
| [5] | "0350AK0010200230000003     | 202301017750000000150  |          |

As you can see the data are not pretty. These data are in “fixed-width format.” Rather than separate each column of data with a delimiter (like a comma or tab), all of the data fields are placed side by side. This is a legacy format that has some benefits. First, there is no need to store delimiters. This is less important these days with cheap data storage, but storing delimiters in a dataset this size requires about 1 Gb just to store a bunch of commas. Instead, the data come with a separate file that describes which columns of text correspond to different columns of data. The following table shows the first five rows of the NIBRS offense segment in the included `NIBRS Records Description updated.xlsx` file.

| Positions | Field Length and Type | Field Name                          |
|-----------|-----------------------|-------------------------------------|
| 1-2       | A2                    | SEGMENT LEVEL                       |
| 3-4       | N2                    | NUMERIC STATE CODE                  |
| 5-13      | A9                    | ORIGINATING AGENCY IDENTIFIER (ORI) |
| 14-25     | A12                   | INCIDENT NUMBER                     |
| 26-33     | A8                    | INCIDENT DATE                       |
| 34-36     | A3                    | UCR OFFENSE CODE                    |
| ...       | ...                   | ...                                 |

The table tells us that for offense segments, the first two characters represent the segment level, characters 3 and 4 capture the state numeric code, characters 5-13 capture the ORI (a unique identifier for a law enforcement agency), and so on. The second column describes the data type, (A)lphanumeric or (N)umeric, and the width (number of characters) of the data field.

The NIBRS data involve several data tables, separate tables for administrative records, offenses, victims, offenders, property, and arrests. The data file interleaves all of these tables. The first two characters of each row indicate which table the row belongs to.

| Segment code | Segment type   |
|--------------|----------------|
| BH           | batch header   |
| 01           | administrative |
| 02           | offenses       |

| Segment code | Segment type        |
|--------------|---------------------|
| 03           | property            |
| 04           | victims             |
| 05           | offender            |
| 06           | arrestee            |
| 07           | Group B arrests     |
| W1           | Incomplete admin    |
| W3           | Incomplete property |
| W6           | Incomplete arrest   |

The W1, W3, and W6 are “window segments” and represent partial reporting. These are relatively rare records and mostly relate to arrests or recovered property connected to offenses that do not appear in the offense segment, most likely because the agency transitioned to NIBRS between the offense and the recovery/arrest.

## 4 The NIBRS format lookup table

I have created a lookup table called `nibrs_format.csv` and included it with the course materials. This table tells R exactly how wide each column is and what type of data it contains for every segment.

```
nibrs_format <- read.csv("NIBRS/nibrs_format.csv")
head(nibrs_format)
```

|   | col_name                          | col_type | col_width | segment |
|---|-----------------------------------|----------|-----------|---------|
| 1 | segment_level                     | c        | 2         | 01      |
| 2 | numeric_state_code                | n        | 2         | 01      |
| 3 | originating_agency_identifier_ori | c        | 9         | 01      |
| 4 | incident_number                   | c        | 12        | 01      |
| 5 | incident_date                     | c        | 8         | 01      |
| 6 | report_date_indicator             | c        | 1         | 01      |

The `nibrs_format` table has four columns:

- `col_name`: the cleaned-up column name we will use in our database
- `col_type`: “c” for character (text) or “n” for numeric
- `col_width`: the number of characters that this column occupies in the fixed-width file
- `segment`: the two-character segment code (01, 02, 03, 04, 05, 06, 07, BH)

Here, for example, are the column specifications for the offense segment.

```
nibrs_format |> filter(segment=="02")
```

|    | col_name                               | col_type | col_width | segment |
|----|----------------------------------------|----------|-----------|---------|
| 1  | segment_level                          | c        | 2         | 02      |
| 2  | numeric_state_code                     | n        | 2         | 02      |
| 3  | originating_agency_identifier_ori      | c        | 9         | 02      |
| 4  | incident_number                        | c        | 12        | 02      |
| 5  | incident_date                          | c        | 8         | 02      |
| 6  | ucr_offense_code                       | c        | 3         | 02      |
| 7  | offense_attempted_completed            | c        | 1         | 02      |
| 8  | suspected_of_using_1                   | c        | 1         | 02      |
| 9  | suspected_of_using_2                   | c        | 1         | 02      |
| 10 | suspected_of_using_3                   | c        | 1         | 02      |
| 11 | location_type                          | c        | 2         | 02      |
| 12 | number_of_premises_entered             | c        | 2         | 02      |
| 13 | method_of_entry                        | c        | 1         | 02      |
| 14 | criminal_activity_1_gang_information_1 | c        | 1         | 02      |
| 15 | criminal_activity_2_gang_information_2 | c        | 1         | 02      |
| 16 | criminal_activity_3                    | c        | 1         | 02      |
| 17 | weapon_force_1                         | c        | 2         | 02      |
| 18 | automatic_indicator_1                  | c        | 1         | 02      |
| 19 | weapon_force_2                         | c        | 3         | 02      |
| 20 | weapon_force_3                         | c        | 3         | 02      |
| 21 | bias_motivation                        | c        | 2         | 02      |

The following code was used to create `nibrs_format.csv` from the Excel documentation file. You do not need to run this code, but it is included here for reference.

```
library(readxl)
library(tidyr)

fmtXL <- read_excel("NIBRS Records Description updated.xlsx",
  sheet = "INCIDENT RECORD",
  range = "A5:D819") |>
  rename(DataField=`Data Field Number`,
    TypeLength=`Type/ Length`)

# Helper to pull format rows for one segment and clean them up
clean_fmt <- function(rows, exclude_pos=NULL) {
  fmt <- fmtXL |>
```

```

    slice(rows) |>
    filter(!is.na(Position))
  if (!is.null(exclude_pos))
    fmt <- fmt |> filter(!(Position %in% exclude_pos))
  data.frame(
    col_name = fmt$Description |>
      strsplit(split=" - ", fixed=TRUE) |>
      sapply(head, n=1) |>
      gsub("[^A-Za-z0-9]+", "_", x=_) |>
      gsub("_+$", "", x=_) |>
      tolower(),
    col_type = fmt$TypeLength |>
      substring(1,1) |>
      recode_values(from=c("A","N"), to=c("c","n")),
    col_width = fmt$TypeLength |>
      substring(2) |>
      as.numeric()
  )
}

# Locate the start of each segment in fmtXL
i02 <- grep('LEVEL "02"', fmtXL$DataField)
i03 <- grep('LEVEL "03"', fmtXL$DataField)
i04 <- grep('LEVEL "04"', fmtXL$DataField)
i05 <- grep('LEVEL "05"', fmtXL$DataField)
i06 <- grep('LEVEL "06"', fmtXL$DataField)
i07 <- grep('LEVEL "07"', fmtXL$DataField)

nibrs_format <- bind_rows(
  clean_fmt(3:(i02-3), "59-88") |> mutate(segment="01"),
  clean_fmt((i02+1):(i03-3), c("38-40","46-48","49-57")) |> mutate(segment="02"),
  clean_fmt((i03+2):(i04-3), c("14-22","20-22","58-102","58-72","103-132")) |> mutate(segment="03"),
  clean_fmt((i04+1):(i05-3), c("37-66","74-77","79-83","84-123")) |> mutate(segment="04"),
  clean_fmt((i05+1):(i06-3)) |> mutate(segment="05"),
  clean_fmt((i06+2):(i07-3), c("61-66","75-104")) |> mutate(segment="06"),
  clean_fmt(-(1:i07), "44-49") |> mutate(segment="07")
)

# The property segment has two columns both named ESTIMATED.QUANTITY;
# the second represents thousandths of the first, so rename it.
nibrs_format <- nibrs_format |>
  mutate(col_name = if_else(

```



```

    segment=="03" &
    col_name=="estimated_quantity" &
    duplicated(paste(segment, col_name)),
    "estimated_quantity_1000ths",
    col_name))

# Shorten the unwieldy type_property_loss_etc name
nibrs_format <- nibrs_format |>
  mutate(col_name = if_else(col_name=="type_property_loss_etc",
                           "type_property_loss",
                           col_name))

# Batch header lives on a different sheet
fmtBH <- read_excel("NIBRS Records Description updated.xlsx", skip=4) |>
  data.frame() |>
  filter(!is.na(Position) &
         !(Position %in% c("106-225", "234-269", "234", "235", "236", "270-284")))

nibrs_format <- bind_rows(
  nibrs_format,
  data.frame(
    col_name = fmtBH$Description |>
      strsplit(split=" - ", fixed=TRUE) |>
      sapply(head, n=1) |>
      gsub("[^A-Za-z0-9]+", "_", x=_) |>
      gsub("_+$", "", x=_) |>
      tolower(),
    col_type = fmtBH$Type..Length |>
      substring(1,1) |>
      recode_values(from=c("A", "N"), to=c("c", "n")),
    col_width = fmtBH$Type..Length |>
      substring(2) |>
      as.numeric(),
    segment = "BH"
  )
)

write_csv(nibrs_format, "NIBRS/nibrs_format.csv")

```

## 5 Setting up the SQLite database

We will create a SQLite database called `nibrs2023.db` to store all the NIBRS segments. Once built, you can query the database from R in any future session without rebuilding it... simply reconnect.

```
# if rebuilding, delete the database if it already exists to start from scratch
if(rebuildDB && file.exists("NIBRS/nibrs2023.db")) file.remove("NIBRS/nibrs2023.db")
# connect to the database (creates the file if it does not exist)
con <- dbConnect(SQLite(), dbname="NIBRS/nibrs2023.db")
```

## 6 Loading the data into SQLite

The master file interleaves all eight segment types into a single enormous file. Our goal is to separate them and store each as its own table in the SQLite database. Here is the strategy.

We will read the master file one million rows at a time. Each batch gets split by the first two characters of each line, which is the segment code. Then, for each segment in that batch, we parse the fixed-width lines directly into a data frame and append it to the appropriate SQLite table. We repeat until the file is exhausted.

The key to making this fast is `I()`. Normally `read_fwf()` expects a file path as its first argument. Wrapping a character vector in `I()`, which stands for “as-is”, tells `read_fwf()` to treat the vector itself as the data rather than as a path to a file. This means we can feed each batch of raw text lines directly to `read_fwf()` without first writing them to an intermediate file. The result is that we read the master file exactly once and write each row to SQLite exactly once, rather than touching the data twice with a set of intermediate files in between.

The other performance trick here is `dbBegin()` and `dbCommit()`. By default, every `dbWriteTable()` call is its own transaction. SQLite flushes its write buffer to disk after every one. With 73 million rows spread across dozens of loop iterations, that amounts to hundreds of individual disk commits and can slow the process to a crawl. `dbBegin()` opens a single transaction that spans the entire loop. SQLite accumulates all the writes in memory and only commits them to disk once when `dbCommit()` is called at the end. This can make bulk loading an order of magnitude faster.

Building the database can take many minutes to run and will produce a `nibrs2023.db` database file that is over 5Gb.

```
# Map segment codes to SQLite table names
seg_tables <- list(
  "01" = "admin",
  "02" = "offense",
```

```

"03" = "property",
"04" = "victim",
"05" = "offender",
"06" = "arrestee",
"07" = "groupb_arrest",
"BH" = "batch_header"
)

# Start fresh - remove any existing tables
for (tbl in unlist(seg_tables)) {
  if (dbExistsTable(con, tbl)) dbRemoveTable(con, tbl)
}

infile <- file("NIBRS/2023_NIBRS_NATIONAL_MASTER_FILE.txt.gz", 'r')
dbBegin(con) # open one transaction for the entire load
linesRead <- 0
while ((length(a <- readLines(infile, n=1000000)) > 0))
{
  linesRead <- linesRead + length(a)
  if(linesRead <= 1e7 || # less than 10M
      linesRead %% 1e7 == 0 || # a multiple of 10M
      length(a) < 1e6) # the last lines read
  {
    message("Lines read: ", format(linesRead, big.mark=",", scientific=FALSE))
  }

  dSplit <- split(a, substring(a, 1, 2))
  # Merge window segments with their corresponding complete segments
  dSplit[["01"]] <- c(dSplit[["01"]], dSplit[["W1"]])
  dSplit[["03"]] <- c(dSplit[["03"]], dSplit[["W3"]])
  dSplit[["06"]] <- c(dSplit[["06"]], dSplit[["W6"]])
  dSplit[c("W1", "W3", "W6")] <- NULL

  for (seg in names(dSplit))
  {
    fmt <- nibrs_format |> filter(segment == seg)

    df <-
      read_fwf(I(dSplit[[seg]]), # parse directly from memory
               col_positions = fwf_widths(fmt$col_width),
               col_type      = paste(fmt$col_type, collapse=""),
               show_col_types = FALSE) |>

```

```

    rename_with(~fmt$col_name) |>
    rename(ori      = originating_agency_identifier_ori,
           segment   = segment_level,
           state     = numeric_state_code)

# Segment-specific adjustments
if (seg == "03")
  df <- df |> mutate(value_of_property = as.numeric(value_of_property))
if (seg == "BH")
  df <- df |> mutate(total_pop = rowSums(across(starts_with("current_pop"))), na.rm=TRUE)
dbWriteTable(con, seg_tables[[seg]], df, append=TRUE)
}
}

dbCommit(con) # flush everything to disk in one shot
close(infile)

```

Let's check what tables are now in our database.

```
dbListTables(con)
```

```

[1] "admin"           "arrestee"        "batch_header"
[4] "circumstance_lookup" "drug_lookup"     "groupb_arrest"
[7] "location_lookup"  "offender"        "offense"
[10] "offense_lookup"   "property"         "property_lookup"
[13] "state_lookup"     "victim"           "weapon_lookup"

```

Our database now has eight data tables, one for each NIBRS segment. (We will add lookup tables later to translate the many codes in the data into readable labels.) Let's look at the first row of the administrative table to confirm the load worked.

```
dbGetQuery(con, "SELECT * FROM admin LIMIT 1")
```

```

segment state      ori incident_number incident_date report_date_indicator
1      01      50 AK0010200          23000003          20230101          <NA>
incident_date_hour total_offense_segments total_victim_segments
1              11              1              1
total_offender_segments total_arrestee_segments city_submission
1              1              0              <NA>
cleared_exceptionally exceptional_clearance_date offense_code_1

```

|   |                       |                |                |                 |                |      |
|---|-----------------------|----------------|----------------|-----------------|----------------|------|
| 1 |                       | N              |                | <NA>            |                | <NA> |
|   | offense_code_2        | offense_code_3 | offense_code_4 | offense_code_5  | offense_code_6 |      |
| 1 | <NA>                  | <NA>           | <NA>           | <NA>            | <NA>           | <NA> |
|   | offense_code_7        | offense_code_8 | offense_code_9 | offense_code_10 |                |      |
| 1 | <NA>                  | <NA>           | <NA>           | <NA>            |                |      |
|   | cargo_theft_indicator |                |                |                 |                |      |
| 1 |                       | N              |                |                 |                |      |

The first incident occurred in Alaska (state 50), reported by the Fairbanks Police Department (ORI AK0010200), on January 1, 2023. The incident number 23000003 is only unique within an agency, so whenever linking to other segments always join on **both** `ori` and `incident_number`.

From here on, all exploration happens through SQL queries. Every time you want to work with your database in a new R session, just reconnect. No need to reload the data:

```
con <- dbConnect(SQLite(), dbname="NIBRS/nibrs2023.db")
```

## 7 Exploring NIBRS with SQL

Now that the data are loaded, we can ask precise questions of our database without ever loading the full dataset into memory. We will cover the key SQL clauses using NIBRS examples. SQL is not case-sensitive, but the convention is to write keywords in ALL CAPS to distinguish them from table and column names.

### 7.1 SELECT, FROM, and WHERE

The three primary SQL clauses are `SELECT`, `FROM`, and `WHERE`. `SELECT` chooses which columns to return. `FROM` names the table where SQL will pull the data. `WHERE` filters rows. Look at the first five offense records.

```
dbGetQuery(con, "
  SELECT ori, incident_number, incident_date, ucr_offense_code
  FROM   offense
  LIMIT  5")
```

|   | ori       | incident_number | incident_date | ucr_offense_code |
|---|-----------|-----------------|---------------|------------------|
| 1 | AK0010200 | 23000003        | 2023-01-01    | 23D              |
| 2 | AK0010200 | 23000005        | 2023-01-01    | 13B              |
| 3 | AK0010200 | 23000011        | 2023-01-02    | 23C              |
| 4 | AK0010200 | 23000013        | 2023-01-02    | 13C              |
| 5 | AK0010200 | 23000014        | 2023-01-02    | 13A              |

The **SELECT** clause includes the three columns that I wish to pull from the table listed in the **FROM** clause, offense. The SQL keyword **LIMIT** is like the R **head()** function and will just show us the first 5 rows of the result rather than the entire offense table. Because there is no **WHERE** clause, this query will pull all the rows from the offense table.

We can use **\*** to select all columns.

```
dbGetQuery(con, "
  SELECT *
  FROM   offense
  LIMIT  3")
```

|   | segment | state | ori       | incident_number | incident_date | ucr_offense_code |
|---|---------|-------|-----------|-----------------|---------------|------------------|
| 1 | 02      | 50    | AK0010200 | 23000003        | 2023-01-01    | 23D              |
| 2 | 02      | 50    | AK0010200 | 23000005        | 2023-01-01    | 13B              |
| 3 | 02      | 50    | AK0010200 | 23000011        | 2023-01-02    | 23C              |

  

|   | offense_attempted_completed | suspected_of_using_1 | suspected_of_using_2 |
|---|-----------------------------|----------------------|----------------------|
| 1 | C                           | N                    | <NA>                 |
| 2 | C                           | N                    | <NA>                 |
| 3 | C                           | N                    | <NA>                 |

  

|   | suspected_of_using_3 | location_type | number_of_premises_entered | method_of_entry |
|---|----------------------|---------------|----------------------------|-----------------|
| 1 | <NA>                 | 04            | <NA>                       | <NA>            |
| 2 | <NA>                 | 20            | <NA>                       | <NA>            |
| 3 | <NA>                 | 12            | <NA>                       | <NA>            |

  

|   | criminal_activity_1_gang_information_1 | criminal_activity_2_gang_information_2 |
|---|----------------------------------------|----------------------------------------|
| 1 | <NA>                                   | <NA>                                   |
| 2 | N                                      | <NA>                                   |
| 3 | <NA>                                   | <NA>                                   |

  

|   | criminal_activity_3 | weapon_force_1 | automatic_indicator_1 | weapon_force_2 |
|---|---------------------|----------------|-----------------------|----------------|
| 1 | <NA>                | <NA>           | <NA>                  | <NA>           |
| 2 | <NA>                | 40             | <NA>                  | <NA>           |
| 3 | <NA>                | <NA>           | <NA>                  | <NA>           |

  

|   | weapon_force_3 | bias_motivation |
|---|----------------|-----------------|
| 1 | <NA>           | 88              |
| 2 | <NA>           | 88              |
| 3 | <NA>           | 88              |

Let's look at the offense record for the Fairbanks, Alaska incident we saw in the administrative segment. The **WHERE** clause can combine multiple conditions with **AND** and **OR**.

```
dbGetQuery(con, "
  SELECT *
```

```
FROM offense
WHERE ori='AK0010200' AND incident_number='23000003'")
```

|   | segment                                | state                                  | ori                        | incident_number      | incident_date | ucr_offense_code |
|---|----------------------------------------|----------------------------------------|----------------------------|----------------------|---------------|------------------|
| 1 | 02                                     | 50                                     | AK0010200                  | 23000003             | 2023-01-01    | 23D              |
|   | offense_attempted_completed            |                                        | suspected_of_using_1       | suspected_of_using_2 |               |                  |
| 1 |                                        |                                        | C                          | N                    |               | <NA>             |
|   | suspected_of_using_3                   | location_type                          | number_of_premises_entered | method_of_entry      |               |                  |
| 1 |                                        | <NA>                                   | 04                         | <NA>                 |               | <NA>             |
|   | criminal_activity_1_gang_information_1 | criminal_activity_2_gang_information_2 |                            |                      |               |                  |
| 1 |                                        | <NA>                                   |                            | <NA>                 |               | <NA>             |
|   | criminal_activity_3                    | weapon_force_1                         | automatic_indicator_1      | weapon_force_2       |               |                  |
| 1 |                                        | <NA>                                   | <NA>                       | <NA>                 |               | <NA>             |
|   | weapon_force_3                         | bias_motivation                        |                            |                      |               |                  |
| 1 |                                        | <NA>                                   |                            | 88                   |               |                  |

The UCR offense code is 23D (theft from a building). `offense_attempted_completed` is C meaning it was a completed crime (rather than attempted). The offender was suspected of using narcotics (`suspected_of_using_1` = N), and the location type is 04 (Church/Synagogue/Temple).

Note that SQL uses a single = to test for equality, unlike R, which uses ==. In SQL, `ori = 'AK0010200'` tests whether `ori` equals that value. Writing == in a SQL query will produce an error. (SQL has no separate assignment operator in a WHERE clause, so there is no ambiguity for the single = to resolve.)

Single quotes are required around text values in SQL WHERE clauses. Double quotes would confuse R, which uses them to delimit the entire query string.

How many offenses were reported to the Philadelphia Police Department (ORI PAPEP0000)?

```
dbGetQuery(con, "
SELECT COUNT(*) AS offense_count
FROM offense
WHERE ori='PAPEP0000'")
```

|   | offense_count |
|---|---------------|
| 1 | 170168        |

COUNT(\*) is an aggregate function that counts all rows. We use AS to give the result a readable column name.

## 7.2 GROUP BY and aggregate functions

GROUP BY groups rows by one or more columns and applies an aggregate function to each group. SQL aggregate functions include COUNT(), SUM(), AVG(), MIN(), and MAX(). The rule to remember is: every column in SELECT that is not inside an aggregate function must appear in GROUP BY.

Let's count offenses by UCR code to find the most commonly reported crime types.

```
dbGetQuery(con, "
  SELECT ucr_offense_code,
         COUNT(*) AS offense_count
  FROM   offense
  GROUP BY ucr_offense_code
  ORDER BY offense_count DESC
  LIMIT 10")
```

|    | ucr_offense_code | offense_count |
|----|------------------|---------------|
| 1  | 13B              | 1935317       |
| 2  | 290              | 1655619       |
| 3  | 23H              | 1421624       |
| 4  | 35A              | 1059452       |
| 5  | 23C              | 966955        |
| 6  | 240              | 892984        |
| 7  | 23F              | 858529        |
| 8  | 220              | 683034        |
| 9  | 13C              | 623972        |
| 10 | 13A              | 605159        |

::: {.callout-important title: What goes in the GROUP BY clause?} *Everything* in the SELECT clause not being aggregated.

What happens if I forget? You will not get an error or warning. SQL will still give you a result, but the SQL standard does not define what that result will be. Most likely it will not be correct. :::

Those UCR codes are hard to interpret. This is a recurring pattern in NIBRS. Many fields are stored as short codes that require a separate lookup table to decode. We will deal with this properly in the next section by loading lookup tables into SQLite and joining them. For now, let's keep exploring.

Let's count offenses by state.



```
dbGetQuery(con, "
  SELECT state,
         COUNT(*) AS offense_count
  FROM   offense
  GROUP BY state
  ORDER BY offense_count DESC
  LIMIT 10")
```

|    | state | offense_count |
|----|-------|---------------|
| 1  | 42    | 1643066       |
| 2  | 4     | 1239084       |
| 3  | 31    | 783107        |
| 4  | 32    | 578580        |
| 5  | 12    | 572438        |
| 6  | 34    | 506654        |
| 7  | 41    | 476048        |
| 8  | 46    | 456304        |
| 9  | 21    | 449002        |
| 10 | 10    | 442508        |

Again we have codes rather than names. We have a `nibrs_state_lookup.csv` file that maps numeric state codes to names. We can load it into R temporarily to decode the top result.

```
a <- read.csv("NIBRS/nibrs_state_lookup.csv")
head(a)
```

|   | state_code | state_abbr | state_name     |
|---|------------|------------|----------------|
| 1 | 50         | AK         | Alaska         |
| 2 | 1          | AL         | Alabama        |
| 3 | 3          | AR         | Arkansas       |
| 4 | 54         | AS         | American Samoa |
| 5 | 2          | AZ         | Arizona        |
| 6 | 4          | CA         | California     |

```
a |> filter(state_code == 42)
```

|   | state_code | state_abbr | state_name |
|---|------------|------------|------------|
| 1 | 42         | TX         | Texas      |

State 42 is Texas, which tends to report more offenses than other states, not necessarily because it has more crime, but because it has very good NIBRS participation across both large and small agencies. Shortly we will load all the lookup tables into SQLite so we can decode codes directly in SQL queries rather than bouncing back to R.

### 7.3 ORDER BY and DISTINCT

ORDER BY sorts the results. Add DESC to sort in descending order (largest first). DISTINCT removes duplicate rows from a result. Let's find the unique set of offense types present in the database.

```
dbGetQuery(con, "
  SELECT DISTINCT ucr_offense_code
  FROM   offense
  ORDER BY ucr_offense_code")
```

|    | ucr_offense_code |
|----|------------------|
| 1  | 09A              |
| 2  | 09B              |
| 3  | 09C              |
| 4  | 100              |
| 5  | 11A              |
| 6  | 11B              |
| 7  | 11C              |
| 8  | 11D              |
| 9  | 120              |
| 10 | 13A              |
| 11 | 13B              |
| 12 | 13C              |
| 13 | 200              |
| 14 | 210              |
| 15 | 220              |
| 16 | 23A              |
| 17 | 23B              |
| 18 | 23C              |
| 19 | 23D              |
| 20 | 23E              |
| 21 | 23F              |
| 22 | 23G              |
| 23 | 23H              |
| 24 | 240              |

|    |     |
|----|-----|
| 25 | 250 |
| 26 | 26A |
| 27 | 26B |
| 28 | 26C |
| 29 | 26D |
| 30 | 26E |
| 31 | 26F |
| 32 | 26G |
| 33 | 26H |
| 34 | 270 |
| 35 | 280 |
| 36 | 290 |
| 37 | 30A |
| 38 | 35A |
| 39 | 35B |
| 40 | 360 |
| 41 | 36A |
| 42 | 36B |
| 43 | 370 |
| 44 | 39A |
| 45 | 39B |
| 46 | 39C |
| 47 | 39D |
| 48 | 40A |
| 49 | 40B |
| 50 | 40C |
| 51 | 49B |
| 52 | 510 |
| 53 | 520 |
| 54 | 526 |
| 55 | 61A |
| 56 | 61B |
| 57 | 620 |
| 58 | 64A |
| 59 | 64B |
| 60 | 720 |

Every offense code in that list corresponds to a crime type in `nibrs_offense_lookup.csv`. Peek at it to get a sense of what is there.

```
read.csv("NIBRS/nibrs_offense_lookup.csv") |> head(10)
```

ucr\_code

crime\_cat

|    |     |                                          |
|----|-----|------------------------------------------|
| 1  | 720 | Animal Cruelty Offenses                  |
| 2  | 200 | Arson                                    |
| 3  | 13A | Assault Offenses                         |
| 4  | 13B | Assault Offenses                         |
| 5  | 13C | Assault Offenses                         |
| 6  | 510 | Bribery                                  |
| 7  | 220 | Burglary/Breaking & Entering             |
| 8  | 250 | Counterfeiting/Forgery                   |
| 9  | 290 | Destruction/Damage/Vandalism of Property |
| 10 | 35A | Drug/Narcotic Offenses                   |
|    |     | crime                                    |
| 1  |     | Animal Cruelty                           |
| 2  |     | Arson                                    |
| 3  |     | Aggravated Assault                       |
| 4  |     | Simple Assault                           |
| 5  |     | Intimidation                             |
| 6  |     | Bribery                                  |
| 7  |     | Burglary/Breaking & Entering             |
| 8  |     | Counterfeiting/Forgery                   |
| 9  |     | Destruction/Damage/Vandalism of Property |
| 10 |     | Drug/Narcotic Violations                 |

Let's explore the property table.

```
dbGetQuery(con, "
SELECT *
FROM   property
LIMIT 5")
```

|   | segment                         | state                              | ori            | incident_number | incident_date | type_property_loss |
|---|---------------------------------|------------------------------------|----------------|-----------------|---------------|--------------------|
| 1 | 03                              | 50                                 | AK0010200      | 23000003        | 20230101      | 7                  |
| 2 | 03                              | 50                                 | AK0010200      | 23000011        | 20230102      | 7                  |
| 3 | 03                              | 50                                 | AK0010200      | 23000014        | 20230102      | 4                  |
| 4 | 03                              | 50                                 | AK0010200      | 23000015        | 20230103      | 5                  |
| 5 | 03                              | 50                                 | AK0010200      | 23000015        | 20230103      | 7                  |
|   | property_description            | value_of_property                  | date_recovered |                 |               |                    |
| 1 |                                 | 75                                 | 150            |                 | <NA>          |                    |
| 2 |                                 | 88                                 | NA             |                 | <NA>          |                    |
| 3 |                                 | 03                                 | 2000           |                 | <NA>          |                    |
| 4 |                                 | 28                                 | 2000           |                 | 20230103      |                    |
| 5 |                                 | 28                                 | 2000           |                 | <NA>          |                    |
|   | number_of_stolen_motor_vehicles | number_of_recovered_motor_vehicles |                |                 |               |                    |

|   |      |      |
|---|------|------|
| 1 | <NA> | <NA> |
| 2 | <NA> | <NA> |
| 3 | <NA> | <NA> |
| 4 | <NA> | 01   |
| 5 | 01   | <NA> |

  

| suspected_drug_type estimated_quantity estimated_quantity_1000ths |      |      |
|-------------------------------------------------------------------|------|------|
| 1                                                                 | <NA> | <NA> |
| 2                                                                 | <NA> | <NA> |
| 3                                                                 | <NA> | <NA> |
| 4                                                                 | <NA> | <NA> |
| 5                                                                 | <NA> | <NA> |

  

| type_measurement drug_involvement_2 drug_involvement_3 |      |      |
|--------------------------------------------------------|------|------|
| 1                                                      | <NA> | <NA> |
| 2                                                      | <NA> | <NA> |
| 3                                                      | <NA> | <NA> |
| 4                                                      | <NA> | <NA> |
| 5                                                      | <NA> | <NA> |

  

| window_ucr_offense_code_1 window_ucr_offense_code_2 window_ucr_offense_code_3 |      |      |
|-------------------------------------------------------------------------------|------|------|
| 1                                                                             | <NA> | <NA> |
| 2                                                                             | <NA> | <NA> |
| 3                                                                             | <NA> | <NA> |
| 4                                                                             | <NA> | <NA> |
| 5                                                                             | <NA> | <NA> |

  

| window_ucr_offense_code_4 window_ucr_offense_code_5 window_ucr_offense_code_6 |      |      |
|-------------------------------------------------------------------------------|------|------|
| 1                                                                             | <NA> | <NA> |
| 2                                                                             | <NA> | <NA> |
| 3                                                                             | <NA> | <NA> |
| 4                                                                             | <NA> | <NA> |
| 5                                                                             | <NA> | <NA> |

  

| window_ucr_offense_code_7 window_ucr_offense_code_8 window_ucr_offense_code_9 |      |      |
|-------------------------------------------------------------------------------|------|------|
| 1                                                                             | <NA> | <NA> |
| 2                                                                             | <NA> | <NA> |
| 3                                                                             | <NA> | <NA> |
| 4                                                                             | <NA> | <NA> |
| 5                                                                             | <NA> | <NA> |

  

| window_ucr_offense_code_10 |      |  |
|----------------------------|------|--|
| 1                          | <NA> |  |
| 2                          | <NA> |  |
| 3                          | <NA> |  |
| 4                          | <NA> |  |
| 5                          | <NA> |  |

These show five records related to property crimes from Fairbanks, Alaska. There are four

unique incident numbers. Note that the last two records relate to the same incident. I have made a lookup table for the codes in `property_description`.

```
read.csv("NIBRS/nibrs_property_lookup.csv") |>
  filter(property_code %in% c(75, 88, 03, 28))
```

|   | property_code | property_type                      |
|---|---------------|------------------------------------|
| 1 | 3             | Automobiles                        |
| 2 | 28            | Recreational Vehicles              |
| 3 | 75            | Portable Electronic Communications |
| 4 | 88            | Pending Inventory                  |

So the first record relates to the theft of what is most likely a mobile phone, the second is unknown, the third is a stolen car, and the last two rows relate to the theft of a recreational vehicle. The `type_property_loss` column explains what exactly happened.

| Code | Description                                                                           |
|------|---------------------------------------------------------------------------------------|
| 1    | None                                                                                  |
| 2    | Burned (includes damage caused in fighting the fire)                                  |
| 3    | Counterfeited/Forged                                                                  |
| 4    | Destroyed/Damaged/Vandalized                                                          |
| 5    | Recovered (to impound property that was previously stolen)                            |
| 6    | Seized (to impound property that was not previously stolen)                           |
| 7    | Stolen/Etc. (includes bribed, defrauded, embezzled, extorted, ransomed, robbed, etc.) |
| 8    | Unknown                                                                               |

So we have 1 stolen mobile phone, 1 stolen mystery item, 1 damaged car, and an recreational vehicle that was stolen (Code 7) and recovered on the same day (Code 5). The data also give the estimated value of the items, but these values can look a little strange, so let's explore those a little more.

`MIN()`, `MAX()`, `AVG()`, and `SUM()` work with numeric columns. Here is the range of reported property values by type of property stolen. The `property_description` column is yet another code field. Code 75, for example, is "Portable Electronic Communications" (i.e., a mobile phone).

```
dbGetQuery(con, "
  SELECT property_description,
         COUNT(*)              AS n,
         MIN(value_of_property) AS min_value,
```

```

        MAX(value_of_property)          AS max_value,
        ROUND(AVG(value_of_property), 0) AS avg_value
FROM    property
WHERE   value_of_property > 1          -- 1 means 'unknown value'
        AND type_property_loss = '7'   -- 7 means stolen
GROUP BY property_description
ORDER BY avg_value DESC
LIMIT 10")

```

|    | property_description | n      | min_value | max_value | avg_value |
|----|----------------------|--------|-----------|-----------|-----------|
| 1  | 29                   | 521    | 2         | 8800000   | 63159     |
| 2  | 05                   | 1067   | 2         | 650000    | 29703     |
| 3  | 01                   | 441    | 2         | 6000000   | 28078     |
| 4  | 37                   | 79821  | 2         | 6800000   | 22769     |
| 5  | 15                   | 24570  | 2         | 3320052   | 18856     |
| 6  | 03                   | 531828 | 2         | 225000000 | 17428     |
| 7  | 33                   | 149    | 2         | 1200000   | 16180     |
| 8  | 73                   | 9304   | 2         | 100000000 | 13885     |
| 9  | 30                   | 329    | 2         | 500000    | 12791     |
| 10 | 31                   | 376    | 2         | 800000    | 11961     |

SQL comments start with --. Anything after -- on a line is ignored by the database engine.

I don't like that scientific notation 1e+09. It could be hiding something important.

```

# "penalize" the printing of scientific notation
options(scipen=999)

dbGetQuery(con, "
    SELECT property_description,
           COUNT(*)              AS n,
           MIN(value_of_property) AS min_value,
           MAX(value_of_property) AS max_value,
           ROUND(AVG(value_of_property), 0) AS avg_value
    FROM    property
    WHERE   value_of_property > 1
           AND type_property_loss = '7'
    GROUP BY property_description
    ORDER BY avg_value DESC
    LIMIT 10")

```

|  | property_description | n | min_value | max_value | avg_value |
|--|----------------------|---|-----------|-----------|-----------|
|--|----------------------|---|-----------|-----------|-----------|

|    |    |        |   |           |       |
|----|----|--------|---|-----------|-------|
| 1  | 29 | 521    | 2 | 8800000   | 63159 |
| 2  | 05 | 1067   | 2 | 650000    | 29703 |
| 3  | 01 | 441    | 2 | 6000000   | 28078 |
| 4  | 37 | 79821  | 2 | 6800000   | 22769 |
| 5  | 15 | 24570  | 2 | 3320052   | 18856 |
| 6  | 03 | 531828 | 2 | 225000000 | 17428 |
| 7  | 33 | 149    | 2 | 1200000   | 16180 |
| 8  | 73 | 9304   | 2 | 100000000 | 13885 |
| 9  | 30 | 329    | 2 | 500000    | 12791 |
| 10 | 31 | 376    | 2 | 800000    | 11961 |

Maximum values of exactly 999999998 are extremely suspicious. The NIBRS documentation says “If the value of the property is unknown, the value will be one (“1”) dollar.” However, it seems that at data entry, 999999998 is getting used to indicate some kind of missing value. It is very common, particularly in older or official data collections to use 9, 99, -9, or other variations of lots of 9s to indicate missing values.

```
dbGetQuery(con, "
  SELECT COUNT(*)      AS n,
         value_of_property
  FROM   property
  WHERE  value_of_property > 1
        AND type_property_loss = 7
  GROUP BY value_of_property
  ORDER BY value_of_property DESC
  LIMIT 10")
```

|    | n | value_of_property |
|----|---|-------------------|
| 1  | 1 | 530795858         |
| 2  | 1 | 400000000         |
| 3  | 1 | 256000000         |
| 4  | 1 | 240160000         |
| 5  | 1 | 225000000         |
| 6  | 1 | 200000000         |
| 7  | 1 | 196000000         |
| 8  | 1 | 180000000         |
| 9  | 1 | 161290000         |
| 10 | 2 | 144000000         |

Indeed there are several hundred cases with values having a lot of 9s. Are these just a few departments using this as a property value? How can I look up the names of the cities associated



with the reports of property values near \$1 billion? I can tell SQL to *join* two tables together, here `property` and `batch_header`, to pull up the city and state. We will cover joins in detail in the next section. For now, just notice that the join lets us bring together information that lives in two separate tables.

```
dbGetQuery(con, "
  SELECT COUNT(*) AS n,
         batch_header.city_name,
         batch_header.state_abbreviation
  FROM   property
        INNER JOIN batch_header
          ON property.ori = batch_header.ori
 WHERE  property.value_of_property >= 999999998
 GROUP BY batch_header.city_name,
         batch_header.state_abbreviation
 ORDER BY n DESC
")
```

```
[1] n              city_name          state_abbreviation
<0 rows> (or 0-length row.names)
```

Now we know that it seems to be largely an issue with four New York police departments.

Let's replace 1, 999999998, and 999999999 with NULL, SQL's missing value indicator.

```
dbExecute(con, "
  UPDATE property
 SET    value_of_property = NULL
 WHERE value_of_property IN (1, 999999998, 999999999)")
```

```
[1] 0
```

Reading raw codes out of results is tedious and error-prone. Joining to a table that holds the city and state was easier than looking up each agency separately. The same idea applies to all the other codes in NIBRS. So let's load all the lookup tables into SQLite so that going forward we can join them directly in our queries.

## 7.4 Joining tables

We got a first taste of joining when we looked up city names above. Now let's look at joins more carefully, because they are central to working with NIBRS.

Working with NIBRS requires connecting information across tables in two distinct ways. First, the NIBRS incident data is spread across multiple segment tables, offenses, victims, offenders, property, and arrestees, and we need to link them by `ori` and `incident_number` to study how these pieces relate to each other. Second, nearly every column in NIBRS is a numeric or short-text code, and we need separate lookup tables to translate those codes into readable labels.

A JOIN is how SQL links two tables together. You name both tables in the `FROM` clause, then use an `ON` clause to specify which columns in each table contain the matching keys. For every row in the left table, SQL finds the matching row(s) in the right table and merges the columns together.

### 7.4.1 Loading lookup tables into SQLite

Rather than decoding codes one at a time in R, we load all the lookup tables into SQLite once. Then we can decode any code with a simple JOIN directly inside a SQL query.

```
lookups <- list(
  offense_lookup      = "NIBRS/nibrs_offense_lookup.csv",
  state_lookup        = "NIBRS/nibrs_state_lookup.csv",
  location_lookup     = "NIBRS/nibrs_location_lookup.csv",
  weapon_lookup       = "NIBRS/nibrs_weapon_lookup.csv",
  property_lookup     = "NIBRS/nibrs_property_lookup.csv",
  drug_lookup         = "NIBRS/nibrs_drug_lookup.csv",
  circumstance_lookup = "NIBRS/nibrs_circumstance_lookup.csv"
)

for (tbl in names(lookups)) {
  if (dbExistsTable(con, tbl)) dbRemoveTable(con, tbl)
  dbWriteTable(con, tbl, read.csv(lookups[[tbl]]))
}
```

```
dbListTables(con)
```

|                           |               |                 |
|---------------------------|---------------|-----------------|
| [1] "admin"               | "arrestee"    | "batch_header"  |
| [4] "circumstance_lookup" | "drug_lookup" | "groupb_arrest" |
| [7] "location_lookup"     | "offender"    | "offense"       |

|      |                  |            |                   |
|------|------------------|------------|-------------------|
| [10] | "offense_lookup" | "property" | "property_lookup" |
| [13] | "state_lookup"   | "victim"   | "weapon_lookup"   |

Our database now has fifteen tables: the eight segment tables we loaded from the master file and the seven lookup tables.

### 7.4.2 INNER JOIN

An **INNER JOIN** returns only rows where the join key exists in **both** tables. Rows with no match in the other table are dropped entirely.

The syntax is:

```
FROM left_table
  INNER JOIN right_table
    ON left_table.key_column = right_table.key_column
```

Let's use **offense\_lookup** to replace UCR codes with readable crime names for the top offenses reported to the Philadelphia Police Department.

```
dbGetQuery(con, "
  SELECT offense_lookup.crime,
         COUNT(*) AS offense_count
  FROM   offense
        INNER JOIN offense_lookup
          ON offense.ucr_offense_code = offense_lookup.ucr_code
 WHERE  offense.ori = 'PAPEP0000'
 GROUP BY offense_lookup.crime
 ORDER BY offense_count DESC
 LIMIT 10")
```

|    | crime                                    | offense_count |
|----|------------------------------------------|---------------|
| 1  | Destruction/Damage/Vandalism of Property | 31068         |
| 2  | Motor Vehicle Theft                      | 23549         |
| 3  | Shoplifting                              | 18197         |
| 4  | Simple Assault                           | 13928         |
| 5  | All Other Larceny                        | 12610         |
| 6  | Theft From Motor Vehicle                 | 10848         |
| 7  | Intimidation                             | 9190          |
| 8  | Aggravated Assault                       | 8139          |
| 9  | Weapon Law Violations                    | 7386          |
| 10 | Burglary/Breaking & Entering             | 5651          |

When a column name could exist in multiple tables (like `ori`), prefix it with the table name, `offense.ori`, so SQL knows exactly where to look. The general practice is to always prefix in JOIN queries for clarity, even when it is technically unambiguous.

Now let's join the `offense` table to `batch_header` to find the cities that report the most offenses. The `batch_header` has one row per agency and the `offense` table has one row per offense. The join brings city and state names into the offense counts.

```
dbGetQuery(con, "
  SELECT batch_header.city_name,
         batch_header.state_abbreviation,
         COUNT(*) AS offense_count
  FROM   offense
        INNER JOIN batch_header
              ON offense.ori = batch_header.ori
  GROUP BY batch_header.city_name, batch_header.state_abbreviation
  ORDER BY offense_count DESC
  LIMIT 10")
```

|    | city_name    | state_abbreviation | offense_count |
|----|--------------|--------------------|---------------|
| 1  | NEW YORK     | NY                 | 560908        |
| 2  | HOUSTON      | TX                 | 345644        |
| 3  | CHICAGO      | IL                 | 244872        |
| 4  | SAN ANTONIO  | TX                 | 177704        |
| 5  | PHILADELPHIA | PA                 | 171121        |
| 6  | LAS VEGAS    | NV                 | 129885        |
| 7  | MEMPHIS      | TN                 | 127246        |
| 8  | DALLAS       | TX                 | 114033        |
| 9  | DETROIT      | MI                 | 84876         |
| 10 | WASHINGTON   | DC                 | 84326         |

Now let's revisit the property value query from earlier. Instead of raw numeric codes we join `property_lookup` to get readable property type names.

```
dbGetQuery(con, "
  SELECT property_lookup.property_type,
         COUNT(*) AS n,
         MIN(value_of_property) AS min_value,
         MAX(value_of_property) AS max_value,
         ROUND(AVG(value_of_property), 0) AS avg_value,
         MEDIAN(value_of_property) AS med_value
  FROM   property
```

```

    INNER JOIN property_lookup
      ON property.property_description = property_lookup.property_code
WHERE  value_of_property IS NOT NULL
      AND type_property_loss = '7'
GROUP BY property_lookup.property_type
ORDER BY med_value DESC
LIMIT 10")

```

|    | property_type                           | n      | min_value | max_value | avg_value |
|----|-----------------------------------------|--------|-----------|-----------|-----------|
| 1  | Buses                                   | 1067   | 2         | 650000    | 29703     |
| 2  | Trucks                                  | 79821  | 2         | 6800000   | 22769     |
| 3  | Automobiles                             | 531828 | 2         | 225000000 | 17428     |
| 4  | Other Motor Vehicles                    | 54578  | 2         | 3223976   | 10354     |
| 5  | Trailers                                | 39120  | 2         | 100000000 | 11638     |
| 6  | Recreational Vehicles                   | 13978  | 2         | 9999999   | 10442     |
| 7  | Heavy Construction/Industrial Equipment | 24570  | 2         | 3320052   | 18856     |
| 8  | Watercraft                              | 3494   | 2         | 1000000   | 11573     |
| 9  | Structures - Single Occupancy Dwellings | 521    | 2         | 8800000   | 63159     |
| 10 | Farm Equipment                          | 7936   | 2         | 6423760   | 11113     |

  

|    | med_value |
|----|-----------|
| 1  | 15000.0   |
| 2  | 12000.0   |
| 3  | 10000.0   |
| 4  | 5000.0    |
| 5  | 4500.0    |
| 6  | 4000.0    |
| 7  | 3500.0    |
| 8  | 2155.5    |
| 9  | 2050.0    |
| 10 | 2000.0    |

### ! Extreme values warrant scrutiny even after cleaning

Even after replacing the known sentinel values, extreme outliers can still hide data entry errors. The single highest `value_of_property` in the dataset — \$225,000,000 — turns out to be a single automobile (`property_description` = 03, `number_of_stolen_motor_vehicles` = 01) reported stolen and recovered from a Castle Hills, Texas (near San Antonio) parking lot at 3am on September 10, 2023 (ORI TX0150300, incident 2023-09-0020). No offender was identified and no arrest was made. The \$225 million valuation on one car is almost certainly a data entry error... someone likely added a few zeros. The NIBRS data contains no quality check preventing an officer

from entering any dollar amount. When computing aggregate property values, consider using the median, capping or excluding implausible outliers, and always inspect the top of the distribution before reporting results.

Where do crimes happen? Joining `location_lookup` answers that immediately.

```
dbGetQuery(con, "
  SELECT location_lookup.location_type,
         COUNT(*) AS offense_count
  FROM   offense
        INNER JOIN location_lookup
          ON offense.location_type = location_lookup.location_code
  GROUP BY location_lookup.location_type
  ORDER BY offense_count DESC
  LIMIT 10")
```

|    | location_type                 | offense_count |
|----|-------------------------------|---------------|
| 1  | Residence/Home                | 5183989       |
| 2  | Highway/Road/Alley            | 2485451       |
| 3  | Parking Lot/Garage            | 1278600       |
| 4  | Other/Unknown                 | 699834        |
| 5  | Department/Discount Store     | 591328        |
| 6  | Grocery/Supermarket           | 322997        |
| 7  | Specialty Store               | 320153        |
| 8  | Convenience Store             | 305186        |
| 9  | Commercial/Office Building    | 297968        |
| 10 | School - Elementary/Secondary | 255924        |

This confirms the conventional wisdom: your home is the most likely place to be victimized, although that is, of course, where you spend much of your time.

What weapons are most commonly involved in aggravated assaults? The offense table stores up to three weapon codes per offense in `weapon_force_1`, `weapon_force_2`, and `weapon_force_3`. `UNION ALL` stacks all three columns into one so we can count across them together, then we join `weapon_lookup` for readable labels.

```
dbGetQuery(con, "
  WITH all_weapons AS (
    SELECT weapon_force_1 AS weapon_code
    FROM offense
    WHERE ucr_offense_code = '13A'
  UNION ALL
```

```

        SELECT weapon_force_2
        FROM offense
        WHERE ucr_offense_code = '13A' AND
              weapon_force_2 IS NOT NULL
    UNION ALL
        SELECT weapon_force_3
        FROM offense
        WHERE ucr_offense_code = '13A' AND
              weapon_force_3 IS NOT NULL
    )
    SELECT weapon_lookup.weapon_type,
           COUNT(*) AS n
    FROM   all_weapons
           INNER JOIN weapon_lookup ON
               all_weapons.weapon_code = weapon_lookup.weapon_code
    GROUP BY weapon_lookup.weapon_type
    ORDER BY n DESC")

```

|    | weapon_type                         | n      |
|----|-------------------------------------|--------|
| 1  | Personal Weapons (hands/feet/teeth) | 135831 |
| 2  | Handgun                             | 108119 |
| 3  | Knife/Cutting Instrument            | 105070 |
| 4  | Other                               | 65058  |
| 5  | Blunt Object                        | 64112  |
| 6  | Firearm (type not stated)           | 60172  |
| 7  | Motor Vehicle/Vessel                | 28792  |
| 8  | Asphyxiation                        | 20061  |
| 9  | None                                | 17828  |
| 10 | Unknown                             | 12683  |
| 11 | Rifle                               | 7002   |
| 12 | Other Firearm                       | 4917   |
| 13 | Shotgun                             | 3333   |
| 14 | Fire/Incendiary Device              | 1172   |
| 15 | Drugs/Narcotics/Sleeping Pills      | 1055   |
| 16 | Poison (including gas)              | 700    |
| 17 | Explosives                          | 271    |

Note the use of `WITH` here. On the fly, I have created a temporary table called `all_weapons` that the `FROM` clause in the main query can use. This is an example of a Common Table Expression (CTE), which we will discuss more later.

### 7.4.3 LEFT JOIN

An INNER JOIN drops any row from the left table that has no match in the right table. A LEFT JOIN keeps all rows from the left table and fills in NULL for any columns from the right table that did not have a match. In our work, LEFT JOIN is almost always the right choice. For example, we have offenses that we want to link to arrestees. If for a particular offense no arrest has been made (yet), then typically we do not want to drop the offense from our dataset. Instead, we just want an NA for arrestee to indicate that no arrest has been made. A left join is the right choice when you do not want to accidentally drop records.

Start by examining the arrestee table.

```
dbGetQuery(con, "SELECT * from arrestee LIMIT 5")
```

|   | segment | state | ori       | incident_number | incident_date |
|---|---------|-------|-----------|-----------------|---------------|
| 1 | 06      | 50    | AK0010200 | 23000038        | 2023-01-03    |
| 2 | 06      | 50    | AK0010200 | 23000041        | 2023-01-03    |
| 3 | 06      | 50    | AK0010200 | 23000047        | 2023-01-04    |
| 4 | 06      | 50    | AK0010200 | 23000059        | 2023-01-05    |
| 5 | 06      | 50    | AK0010200 | 23000066        | 2023-01-05    |

  

|   | arrestee_sequence_number | arrest_transaction_number | arrest_date | type_of_arrest |
|---|--------------------------|---------------------------|-------------|----------------|
| 1 |                          | 1                         | 50912       | 2023-01-03     |
| 2 |                          | 1                         | 50913       | 2023-01-03     |
| 3 |                          | 1                         | 50918       | 2023-01-04     |
| 4 |                          | 1                         | 50920       | 2023-01-05     |
| 5 |                          | 1                         | 51002       | 2023-01-25     |

  

|   | multiple_arrestee_segments_indicator | ucr_arrest_offense_code | weapon_1 |
|---|--------------------------------------|-------------------------|----------|
| 1 | N                                    | 240                     | 01       |
| 2 | N                                    | 13B                     | 01       |
| 3 | N                                    | 23C                     | 01       |
| 4 | N                                    | 35A                     | 01       |
| 5 | M                                    | 23C                     | 01       |

  

|   | automatic_indicator_1 | weapon_2 | age_of_arrestee | sex_of_arrestee |
|---|-----------------------|----------|-----------------|-----------------|
| 1 | <NA>                  | <NA>     | 26              | M               |
| 2 | <NA>                  | <NA>     | 11              | F               |
| 3 | <NA>                  | <NA>     | 31              | M               |
| 4 | <NA>                  | <NA>     | 23              | M               |
| 5 | <NA>                  | <NA>     | 30              | M               |

  

|   | race_of_arrestee | ethnicity_of_arrestee | resident_status_of_arrestee |
|---|------------------|-----------------------|-----------------------------|
| 1 | I                | N                     | R                           |
| 2 | W                | N                     | R                           |
| 3 | B                | N                     | R                           |



|   | W                                | N                         | R                         |
|---|----------------------------------|---------------------------|---------------------------|
| 4 | W                                | N                         | R                         |
| 5 | W                                | N                         | R                         |
|   | disposition_of_arrestee_under_18 | window_clearance_flag     |                           |
| 1 | <NA>                             | <NA>                      |                           |
| 2 | R                                | <NA>                      |                           |
| 3 | <NA>                             | <NA>                      |                           |
| 4 | <NA>                             | <NA>                      |                           |
| 5 | <NA>                             | <NA>                      |                           |
|   | window_ucr_offense_code_1        | window_ucr_offense_code_2 | window_ucr_offense_code_3 |
| 1 | <NA>                             | <NA>                      | <NA>                      |
| 2 | <NA>                             | <NA>                      | <NA>                      |
| 3 | <NA>                             | <NA>                      | <NA>                      |
| 4 | <NA>                             | <NA>                      | <NA>                      |
| 5 | <NA>                             | <NA>                      | <NA>                      |
|   | window_ucr_offense_code_4        | window_ucr_offense_code_5 | window_ucr_offense_code_6 |
| 1 | <NA>                             | <NA>                      | <NA>                      |
| 2 | <NA>                             | <NA>                      | <NA>                      |
| 3 | <NA>                             | <NA>                      | <NA>                      |
| 4 | <NA>                             | <NA>                      | <NA>                      |
| 5 | <NA>                             | <NA>                      | <NA>                      |
|   | window_ucr_offense_code_7        | window_ucr_offense_code_8 | window_ucr_offense_code_9 |
| 1 | <NA>                             | <NA>                      | <NA>                      |
| 2 | <NA>                             | <NA>                      | <NA>                      |
| 3 | <NA>                             | <NA>                      | <NA>                      |
| 4 | <NA>                             | <NA>                      | <NA>                      |
| 5 | <NA>                             | <NA>                      | <NA>                      |
|   | window_ucr_offense_code_10       |                           |                           |
| 1 | <NA>                             |                           |                           |
| 2 | <NA>                             |                           |                           |
| 3 | <NA>                             |                           |                           |
| 4 | <NA>                             |                           |                           |
| 5 | <NA>                             |                           |                           |

These show records of five arrests that Fairbanks officers made in January 2023. The data include age, sex, race, and ethnicity of the arrestee. Four of these arrests were “on-view” (`type_of_arrest = O`) and one was taken into custody based on a warrant or prior report (`type_of_arrest = T`). A third option would be a summons or citation (`type_of_arrest = S`).

There can be multiple arrests associated with the same incident. Here’s an incident in Alabama in which two crimes occurred.

```
dbGetQuery(con, "
    SELECT ori, incident_number, ucr_offense_code,
           offense_lookup.crime
    FROM offense
    LEFT JOIN offense_lookup
    ON offense.ucr_offense_code = offense_lookup.ucr_code
    WHERE ori='AL0010000' AND incident_number='0EA8CD847E02'")
```

|   | ori       | incident_number | ucr_offense_code | crime                     |
|---|-----------|-----------------|------------------|---------------------------|
| 1 | AL0010000 | 0EA8CD847E02    | 35A              | Drug/Narcotic Violations  |
| 2 | AL0010000 | 0EA8CD847E02    | 35B              | Drug Equipment Violations |

The report lists three offenders.

```
dbGetQuery(con, "
    SELECT ori, incident_number, offender_sequence_number,
           age_of_offender, sex_of_offender,
           race_of_offender, ethnicity_of_offender
    FROM offender
    WHERE ori='AL0010000' AND incident_number='0EA8CD847E02'")
```

|   | ori       | incident_number | offender_sequence_number | age_of_offender |
|---|-----------|-----------------|--------------------------|-----------------|
| 1 | AL0010000 | 0EA8CD847E02    | 1                        | 53              |
| 2 | AL0010000 | 0EA8CD847E02    | 2                        | 39              |
| 3 | AL0010000 | 0EA8CD847E02    | 5                        | 38              |

  

|   | sex_of_offender | race_of_offender | ethnicity_of_offender |
|---|-----------------|------------------|-----------------------|
| 1 | M               | W                | N                     |
| 2 | M               | W                | N                     |
| 3 | F               | W                | N                     |

And all three were arrested. Note that offender sequence numbers may not line up with the arrest sequence numbers.

```
dbGetQuery(con, "
    SELECT ori, incident_number, arrestee_sequence_number,
           ucr_arrest_offense_code,
           age_of_arrestee, race_of_arrestee, ethnicity_of_arrestee
    FROM arrestee
    WHERE ori='AL0010000' AND incident_number='0EA8CD847E02'")
```

|   | ori       | incident_number | arrestee_sequence_number | ucr_arrest_offense_code |
|---|-----------|-----------------|--------------------------|-------------------------|
| 1 | AL0010000 | 0EA8CD847E02    | 1                        | 35A                     |
| 2 | AL0010000 | 0EA8CD847E02    | 5                        | 35A                     |
| 3 | AL0010000 | 0EA8CD847E02    | 6                        | 35B                     |

  

|   | age_of_arrestee | race_of_arrestee | ethnicity_of_arrestee |
|---|-----------------|------------------|-----------------------|
| 1 | 39              | W                | N                     |
| 2 | 53              | W                | N                     |
| 3 | 38              | W                | N                     |

Now that we have some familiarity with the arrestee table, let's compute the percentage of burglaries (UCR code 220, completed) that resulted in an arrest. Not all incidents will have a matching row in the `arrestee` table, so we use a `LEFT JOIN` to keep all burglaries whether or not an arrest was made.

```
dbGetQuery(con, "
  SELECT offense.incident_date,
         offense.ori,
         offense.incident_number,
         arr.ori           AS arrest_ori,
         arr.incident_number AS arrest_incident_number
  FROM   offense
        LEFT JOIN (SELECT DISTINCT ori, incident_number
                   FROM   arrestee) AS arr
        ON offense.ori = arr.ori
         AND offense.incident_number = arr.incident_number
 WHERE  offense.ucr_offense_code = '220'
        AND offense.offense_attempted_completed = 'C'
 LIMIT 10")
```

|    | incident_date | ori       | incident_number | arrest_ori | arrest_incident_number |
|----|---------------|-----------|-----------------|------------|------------------------|
| 1  | 2023-01-03    | AK0010200 | 23000036        | <NA>       | <NA>                   |
| 2  | 2023-01-05    | AK0010200 | 23000071        | <NA>       | <NA>                   |
| 3  | 2023-01-06    | AK0010200 | 23000085        | <NA>       | <NA>                   |
| 4  | 2023-01-09    | AK0010200 | 23000122        | <NA>       | <NA>                   |
| 5  | 2023-01-09    | AK0010200 | 23000138        | <NA>       | <NA>                   |
| 6  | 2023-01-04    | AK0010200 | 23000171        | <NA>       | <NA>                   |
| 7  | 2023-01-13    | AK0010200 | 23000208        | <NA>       | <NA>                   |
| 8  | 2023-01-14    | AK0010200 | 23000214        | AK0010200  | 23000214               |
| 9  | 2023-01-15    | AK0010200 | 23000232        | <NA>       | <NA>                   |
| 10 | 2023-01-15    | AK0010200 | 23000235        | <NA>       | <NA>                   |

Note that for all but one, the arrest ORI and incident numbers are `NA` indicating that the left join could not find an ORI/incident number match over in the `arrestee` table.

One offense can result in multiple arrests. Note the “subquery” used inside the left join. If there are multiple arrests for an incident, this subquery reduces them to a single row with `DISTINCT`. Otherwise, if one burglary resulted in 10 arrests, then that incident would get counted 10 times. As written, each offense gets one row and will check whether there are any associated arrests.

Now let’s count them up. Note that SQL evaluates all expressions in a `SELECT` clause simultaneously, so an alias defined in one column cannot be referenced by another column in the same `SELECT`. We cannot write `ROUND(100.0 * total_arrests / total_burglaries, 1)` in the same `SELECT` that defines `total_arrests`... SQL does not yet know what `total_arrests` is. A Common Table Expression (CTE) solves this by computing the counts first and dividing in a second step.

```
dbGetQuery(con, "
  WITH burglary_counts AS (
    SELECT COUNT(*) AS total_burglaries,
           SUM(CASE WHEN arr.ori IS NOT NULL
                    THEN 1 ELSE 0 END) AS total_arrests
    FROM   offense
          LEFT JOIN (SELECT DISTINCT ori, incident_number
                    FROM   arrestee) AS arr
          ON offense.ori = arr.ori
          AND offense.incident_number = arr.incident_number
    WHERE  offense.ucr_offense_code = '220'
          AND offense.offense_attempted_completed = 'C'
  )
  SELECT total_burglaries,
         total_arrests,
         ROUND(100.0 * total_arrests / total_burglaries, 1) AS pct_arrested
  FROM   burglary_counts")
```

|   | total_burglaries | total_arrests | pct_arrested |
|---|------------------|---------------|--------------|
| 1 | 637882           | 86561         | 13.6         |

`CASE WHEN ... THEN ... ELSE ... END` is SQL’s version of an if-else expression. Here, if `arr.ori IS NOT NULL` (meaning a match was found in the arrestee table), we add 1; otherwise we add 0. Summing those 1s gives the number of arrests, and dividing by the total count gives the clearance rate.

## 7.5 Date functions

SQLite stores dates as plain text. NIBRS incident dates arrive in `YYYYMMDD` format... no hyphens to save on storage. Let’s confirm:

```
dbGetQuery(con, "
  SELECT incident_date
  FROM    offense
  ORDER BY RANDOM()
  LIMIT 10")
```

|    | incident_date |
|----|---------------|
| 1  | 2023-11-05    |
| 2  | 2023-01-25    |
| 3  | 2023-10-26    |
| 4  | 2023-10-05    |
| 5  | 2023-06-15    |
| 6  | 2023-12-18    |
| 7  | 2023-05-30    |
| 8  | 2023-05-10    |
| 9  | 2023-01-10    |
| 10 | 2023-05-16    |

SQLite's date functions (`STRFTIME()`, `DATE()`, etc.) expect the standard `YYYY-MM-DD` format, so we need to reformat `incident_date` before we can use them. The `UPDATE` statement edits rows in place. We use `SUBSTR()` to pull apart the year, month, and day and reassemble them with hyphens. The `||` operator concatenates strings in SQL.

```
dbExecute(con, "
  UPDATE offense
  SET    incident_date = SUBSTR(incident_date, 1, 4) || '-' ||
                        SUBSTR(incident_date, 5, 2) || '-' ||
                        SUBSTR(incident_date, 7, 2)
  WHERE  incident_date NOT LIKE '____-__-__')"
```

```
[1] 0
```

The `WHERE incident_date NOT LIKE '____-__-__'` clause is a safeguard. Each `_` matches any single character, so the pattern matches dates already in `YYYY-MM-DD` format. This means re-running the chunk will not corrupt already-formatted dates. The same fix applies to `incident_date` and `arrest_date` in the `arrestee` table.

```
dbExecute(con, "
  UPDATE arrestee
  SET    incident_date = SUBSTR(incident_date, 1, 4) || '-' ||
```

```

        SUBSTR(incident_date, 5, 2) || '-' ||
        SUBSTR(incident_date, 7, 2)
WHERE incident_date NOT LIKE '____-__-__')

```

[1] 0

```

dbExecute(con, "
UPDATE arrestee
SET    arrest_date = SUBSTR(arrest_date, 1, 4) || '-' ||
        SUBSTR(arrest_date, 5, 2) || '-' ||
        SUBSTR(arrest_date, 7, 2)
WHERE arrest_date NOT LIKE '____-__-__')

```

[1] 0

Now STRFTIME() works directly. It takes a format string and a date column, and returns the requested part as text. Common format codes: %Y (4-digit year), %m (month 01–12), %d (day), %w (day of week, 0=Sunday).

Let's count offenses by month to look for seasonal patterns.

```

dbGetQuery(con, "
SELECT STRFTIME('%Y-%m', incident_date) AS year_month,
       COUNT(*)                        AS offense_count
FROM   offense
GROUP BY year_month
ORDER BY year_month")

```

|    | year_month | offense_count |
|----|------------|---------------|
| 1  | 2023-01    | 1170426       |
| 2  | 2023-02    | 1050629       |
| 3  | 2023-03    | 1167315       |
| 4  | 2023-04    | 1160311       |
| 5  | 2023-05    | 1249627       |
| 6  | 2023-06    | 1202231       |
| 7  | 2023-07    | 1243591       |
| 8  | 2023-08    | 1240191       |
| 9  | 2023-09    | 1172658       |
| 10 | 2023-10    | 1186363       |
| 11 | 2023-11    | 1078670       |
| 12 | 2023-12    | 1072324       |

Now let's look at whether the clearance rate varies by the month in which the offense occurred. Remember that the March 15 cutoff means crimes committed late in the year have less time to produce a recorded arrest.

```
dbGetQuery(con, "
  WITH clearance AS (
    SELECT STRFTIME('%m', offense.incident_date) AS month,
           COUNT(*) AS total,
           SUM(CASE WHEN arr.ori IS NOT NULL
                    THEN 1 ELSE 0 END) AS arrests
    FROM   offense
    LEFT JOIN (SELECT DISTINCT ori, incident_number
               FROM   arrestee) AS arr
    ON offense.ori = arr.ori
    AND offense.incident_number = arr.incident_number
    GROUP BY month
  )
  SELECT month,
         total,
         arrests,
         ROUND(100.0 * arrests / total, 1) AS pct_arrested
  FROM   clearance
  ORDER BY month")
```

|    | month | total   | arrests | pct_arrested |
|----|-------|---------|---------|--------------|
| 1  | 01    | 1170426 | 321943  | 27.5         |
| 2  | 02    | 1050629 | 291393  | 27.7         |
| 3  | 03    | 1167315 | 324924  | 27.8         |
| 4  | 04    | 1160311 | 321251  | 27.7         |
| 5  | 05    | 1249627 | 340226  | 27.2         |
| 6  | 06    | 1202231 | 323421  | 26.9         |
| 7  | 07    | 1243591 | 333871  | 26.8         |
| 8  | 08    | 1240191 | 333782  | 26.9         |
| 9  | 09    | 1172658 | 317183  | 27.0         |
| 10 | 10    | 1186363 | 312047  | 26.3         |
| 11 | 11    | 1078670 | 289840  | 26.9         |
| 12 | 12    | 1072324 | 288143  | 26.9         |

The clearance rate drops substantially for crimes committed late in the year, not because those crimes are actually less likely to be solved, but because there is less time between the crime and the NIBRS submission cutoff for an arrest to be recorded.

The March 15 cutoff makes raw clearance rates hard to compare. A homicide in January has over a year for an arrest to appear in the data, while a homicide in December has only a few months. To put every crime on equal footing, we can ask a more precise question: was there an arrest *within 60 days* of the offense? Every offense in the year has a full 60-day window to be cleared.

To compute the time between two dates, SQLite provides `JULIANDAY()`, which converts a date to the number of days since a fixed reference point (noon on November 24, 4714 BC, the start of the “Julian day” used by astronomers). Subtracting one Julian day from another gives the number of days between two dates. Let’s look at the time from offense to first arrest for a few homicides (UCR code 09A, murder and non-negligent manslaughter). Because the `arrestee` table can hold several arrests per incident, we first collapse it to the earliest arrest date per incident with `MIN(arrest_date)`.

```
dbGetQuery(con, "
  SELECT offense.ori,
         offense.incident_number,
         offense.incident_date,
         arr.first_arrest,
         JULIANDAY(arr.first_arrest) - JULIANDAY(offense.incident_date) AS days_to_arrest
  FROM   offense
        INNER JOIN (SELECT ori, incident_number, MIN(arrest_date) AS first_arrest
                    FROM   arrestee
                    GROUP BY ori, incident_number) AS arr
        ON offense.ori = arr.ori
        AND offense.incident_number = arr.incident_number
  WHERE  offense.ucr_offense_code = '09A'
  LIMIT 10")
```

|    | ori       | incident_number | incident_date | first_arrest | days_to_arrest |
|----|-----------|-----------------|---------------|--------------|----------------|
| 1  | AK0010200 | 23001511        | 2023-04-23    | 2023-04-23   | 0              |
| 2  | AK0010300 | 202307060076    | 2023-07-06    | 2023-07-07   | 1              |
| 3  | AK0011600 | 23-003432       | 2023-06-14    | 2023-06-15   | 1              |
| 4  | AK0012300 | 2302-0173       | 2023-02-12    | 2023-02-12   | 0              |
| 5  | AKAST0100 | AK23006543      | 2023-01-20    | 2023-01-20   | 0              |
| 6  | AKAST0100 | AK23023794      | 2023-03-09    | 2023-03-09   | 0              |
| 7  | AKAST0100 | AK23027668      | 2023-03-21    | 2023-03-21   | 0              |
| 8  | AKAST0100 | AK23029139      | 2023-03-24    | 2023-03-24   | 0              |
| 9  | AKAST0100 | AK23032044A     | 2023-04-01    | 2023-04-02   | 1              |
| 10 | AKAST0100 | AK23060497      | 2023-06-14    | 2023-06-16   | 2              |

Now let’s compute the homicide clearance rate by state, where “cleared” means an arrest occurred within 60 days of the offense. We use a `LEFT JOIN` so that homicides with no arrest



at all still count in the denominator, and we join `state_lookup` to label the states. Note that `days_to_arrest` is NULL whenever there was no arrest, and any comparison with NULL (like `NULL <= 60`) is not TRUE, so those incidents correctly fall outside the cleared count.

```
dbGetQuery(con, "
  WITH homicide_clearance AS (
    SELECT offense.state,
           JULIANDAY(arr.first_arrest) - JULIANDAY(offense.incident_date) AS days_to_arrest
    FROM   offense
    LEFT JOIN (SELECT ori, incident_number, MIN(arrest_date) AS first_arrest
              FROM   arrestee
              GROUP BY ori, incident_number) AS arr
    ON offense.ori = arr.ori
    AND offense.incident_number = arr.incident_number
    WHERE  offense.ucr_offense_code = '09A'
  )
  SELECT state_lookup.state_name,
         COUNT(*) AS homicides,
         SUM(CASE WHEN days_to_arrest <= 60
                  THEN 1 ELSE 0 END) AS cleared_60,
         ROUND(100.0 * SUM(CASE WHEN days_to_arrest <= 60
                                THEN 1 ELSE 0 END) / COUNT(*), 1) AS pct_cleared_60
  FROM   homicide_clearance
  INNER JOIN state_lookup
    ON CAST(homicide_clearance.state AS INTEGER) = state_lookup.state_code
  GROUP BY state_lookup.state_name
  HAVING homicides >= 50
  ORDER BY pct_cleared_60 DESC")
```

|    | state_name     | homicides | cleared_60 | pct_cleared_60 |
|----|----------------|-----------|------------|----------------|
| 1  | Utah           | 55        | 40         | 72.7           |
| 2  | Iowa           | 76        | 54         | 71.1           |
| 3  | Minnesota      | 168       | 111        | 66.1           |
| 4  | Mississippi    | 102       | 62         | 60.8           |
| 5  | Nevada         | 204       | 122        | 59.8           |
| 6  | Arizona        | 191       | 113        | 59.2           |
| 7  | South Carolina | 415       | 245        | 59.0           |
| 8  | Kansas         | 120       | 66         | 55.0           |
| 9  | Colorado       | 275       | 151        | 54.9           |
| 10 | California     | 842       | 438        | 52.0           |
| 11 | New York       | 495       | 256        | 51.7           |
| 12 | Wisconsin      | 259       | 131        | 50.6           |

|    |                      |      |     |      |
|----|----------------------|------|-----|------|
| 13 | Arkansas             | 241  | 120 | 49.8 |
| 14 | Virginia             | 489  | 239 | 48.9 |
| 15 | Oklahoma             | 227  | 111 | 48.9 |
| 16 | Oregon               | 174  | 84  | 48.3 |
| 17 | Alabama              | 423  | 198 | 46.8 |
| 18 | Louisiana            | 302  | 139 | 46.0 |
| 19 | Washington           | 333  | 147 | 44.1 |
| 20 | Texas                | 1718 | 758 | 44.1 |
| 21 | West Virginia        | 73   | 32  | 43.8 |
| 22 | North Carolina       | 735  | 320 | 43.5 |
| 23 | Missouri             | 528  | 229 | 43.4 |
| 24 | Massachusetts        | 133  | 56  | 42.1 |
| 25 | Kentucky             | 255  | 107 | 42.0 |
| 26 | Michigan             | 554  | 213 | 38.4 |
| 27 | Georgia              | 641  | 246 | 38.4 |
| 28 | Pennsylvania         | 549  | 208 | 37.9 |
| 29 | Indiana              | 346  | 125 | 36.1 |
| 30 | Florida              | 418  | 141 | 33.7 |
| 31 | New Jersey           | 152  | 51  | 33.6 |
| 32 | Ohio                 | 579  | 184 | 31.8 |
| 33 | Maryland             | 464  | 144 | 31.0 |
| 34 | Connecticut          | 127  | 39  | 30.7 |
| 35 | Tennessee            | 646  | 195 | 30.2 |
| 36 | Illinois             | 747  | 197 | 26.4 |
| 37 | District of Columbia | 248  | 62  | 25.0 |
| 38 | New Mexico           | 199  | 39  | 19.6 |

The **HAVING** clause is new here. It filters *groups* after aggregation, whereas **WHERE** filters individual rows *before* aggregation. The distinction matters because of when each clause is evaluated. SQL processes a query in stages: it first uses **WHERE** to decide which rows to keep, then **GROUP BY** collapses those rows into groups and computes the aggregates (**COUNT(\*)**, **SUM(...)**, and so on), and only then does **HAVING** get to filter based on those computed aggregates. At the moment **WHERE** runs, the group counts do not exist yet, so a condition like `homicides >= 50` could not possibly be evaluated there.

That ordering gives us a simple rule of thumb: if your filter condition involves an aggregate function, it must go in **HAVING** and if it involves only raw column values, it goes in **WHERE**. A condition like `offense.ucr_offense_code = '09A'` looks only at raw column values, so it belongs in **WHERE** (and we put it there, inside the CTE). A condition like `COUNT(*) >= 50` depends on an aggregate, so it must go in **HAVING**. As a bonus, filtering in **WHERE** is also faster whenever it is possible, because it discards rows early, before the database does the work of grouping and aggregating them, so prefer **WHERE** for any condition that does not require an aggregate.

Here we use `HAVING homicides >= 50` to keep only states with at least 50 homicides so that the percentages are not dominated by states with just a handful of cases. The resulting 60-day clearance rates are far more comparable across states than a raw count of arrests would be, since every homicide had the same 60-day window in which to be cleared.

Let's use a CTE to find the states with the largest populations not yet covered by NIBRS. We will compute total population by state, split by whether the agency reports to NIBRS, then calculate what percentage of each state's population is covered.

One important SQL gotcha: agencies that do not report to NIBRS have `agency_nibrs_flag = NULL`. In SQL, any comparison with `NULL` (including `NULL != 'A'`) evaluates to `NULL`, not `TRUE`. So the condition `agency_nibrs_flag != 'A'` would silently exclude all non-reporting agencies rather than selecting them. The correct test is `agency_nibrs_flag IS NULL`.

```
dbGetQuery(con, "
WITH -- tabulate state population by state x NIBRS reporting
    pop_by_state AS (
        SELECT state_abbreviation,
               agency_nibrs_flag,
               SUM(total_pop) AS pop
        FROM   batch_header
        WHERE  covered_by_ori IS NULL -- exclude those reporting through other
        GROUP BY state_abbreviation, agency_nibrs_flag)
SELECT state_abbreviation,
       MAX(CASE WHEN agency_nibrs_flag='A' THEN pop ELSE 0 END)      AS in_nibrs,
       MAX(CASE WHEN agency_nibrs_flag IS NULL THEN pop ELSE 0 END) AS not_in_nibrs,
       ROUND(100.0 * MAX(CASE WHEN agency_nibrs_flag='A' THEN pop ELSE 0 END) /
             (MAX(CASE WHEN agency_nibrs_flag='A' THEN pop ELSE 0 END) +
              MAX(CASE WHEN agency_nibrs_flag IS NULL THEN pop ELSE 0 END)), 0) AS pct_cov
FROM   pop_by_state
GROUP BY state_abbreviation
ORDER BY pct_covered")
```

|   | state_abbreviation | in_nibrs | not_in_nibrs | pct_covered |
|---|--------------------|----------|--------------|-------------|
| 1 | PR                 | 0        | 3205691      | 0           |
| 2 | VI                 | 0        | 83265        | 0           |
| 3 | FL                 | 9583627  | 13027099     | 42          |
| 4 | PA                 | 5605372  | 7356311      | 43          |
| 5 | AK                 | 445330   | 288076       | 61          |
| 6 | AZ                 | 4664949  | 2766395      | 63          |
| 7 | CA                 | 25034801 | 13930392     | 64          |
| 8 | MS                 | 1905988  | 1033702      | 65          |
| 9 | NJ                 | 6351608  | 2939233      | 68          |

|    |    |          |         |     |
|----|----|----------|---------|-----|
| 10 | NY | 13614572 | 5956644 | 70  |
| 11 | LA | 3546505  | 1027244 | 78  |
| 12 | IN | 6005304  | 856895  | 88  |
| 13 | WY | 516895   | 67162   | 89  |
| 14 | NM | 1913130  | 201241  | 90  |
| 15 | IL | 11358535 | 1191154 | 91  |
| 16 | AL | 4721997  | 386471  | 92  |
| 17 | GA | 10299315 | 729912  | 93  |
| 18 | KS | 2788431  | 152115  | 95  |
| 19 | NC | 10601690 | 233801  | 98  |
| 20 | WI | 5770024  | 140931  | 98  |
| 21 | MD | 6108229  | 72024   | 99  |
| 22 | MO | 6128867  | 67289   | 99  |
| 23 | OH | 11668804 | 117131  | 99  |
| 24 | TX | 30286412 | 216889  | 99  |
| 25 | AR | 3058001  | 9731    | 100 |
| 26 | CO | 5876374  | 1236    | 100 |
| 27 | CT | 3617176  | 0       | 100 |
| 28 | DC | 678972   | 0       | 100 |
| 29 | DE | 1031890  | 0       | 100 |
| 30 | GM | 153836   | 0       | 100 |
| 31 | HI | 1435138  | 0       | 100 |
| 32 | IA | 3204646  | 2358    | 100 |
| 33 | ID | 1964726  | 0       | 100 |
| 34 | KY | 4526154  | 0       | 100 |
| 35 | MA | 6990737  | 10662   | 100 |
| 36 | ME | 1395722  | 0       | 100 |
| 37 | MI | 10031379 | 5882    | 100 |
| 38 | MN | 5733978  | 3937    | 100 |
| 39 | MT | 1132812  | 0       | 100 |
| 40 | NB | 1977720  | 659     | 100 |
| 41 | ND | 783926   | 0       | 100 |
| 42 | NH | 1400376  | 1678    | 100 |
| 43 | NV | 3181298  | 12878   | 100 |
| 44 | OK | 4053824  | 0       | 100 |
| 45 | OR | 4223654  | 9704    | 100 |
| 46 | RI | 1094548  | 1414    | 100 |
| 47 | SC | 5373555  | 0       | 100 |
| 48 | SD | 919318   | 0       | 100 |
| 49 | TN | 7126489  | 0       | 100 |
| 50 | UT | 3414368  | 3366    | 100 |
| 51 | VA | 8715698  | 0       | 100 |
| 52 | VT | 647464   | 0       | 100 |

|    |    |         |     |     |
|----|----|---------|-----|-----|
| 53 | WA | 7812880 | 0   | 100 |
| 54 | WV | 1769732 | 339 | 100 |

The states with the lowest NIBRS coverage tend to include Florida, Pennsylvania, California, and New York, all very populous states. This is an important limitation to keep in mind when interpreting national statistics from NIBRS.

You can use multiple CTEs in one `WITH` clause, separating them with commas. The query below uses two CTEs to identify the agency with the most shootings (aggravated assault, UCR 13A, completed, firearm involved), then joins in the agency name from the batch header.

```
dbGetQuery(con, "
  WITH -- tabulate shootings by ORI
    shootings AS (
      SELECT ori, COUNT(*) AS shooting_count
      FROM   offense
      WHERE  ucr_offense_code = '13A'
            AND offense_attempted_completed = 'C'
            AND (weapon_force_1 IN ('11','12','13','14','15') OR
                  weapon_force_2 IN ('11','12','13','14','15') OR
                  weapon_force_3 IN ('11','12','13','14','15'))
      GROUP BY ori
    ),
    top_agency AS ( -- which ORI reports the most shootings?
      SELECT ori, shooting_count
      FROM   shootings
      ORDER BY shooting_count DESC
      LIMIT 1
    )
  SELECT batch_header.city_name,
         batch_header.state_abbreviation,
         top_agency.shooting_count
  FROM   top_agency
  INNER JOIN batch_header ON top_agency.ori = batch_header.ori")
```

|   | city_name | state_abbreviation | shooting_count |
|---|-----------|--------------------|----------------|
| 1 | MEMPHIS   | TN                 | 5697           |

## 7.6 Window functions

So far we have not touched the `victim` table. Let's peek at the first few rows.

```
dbGetQuery(con, "
SELECT *
FROM victim
LIMIT 5")
```

|   | segment | state | ori       | incident_number | incident_date | victim_sequence_number |
|---|---------|-------|-----------|-----------------|---------------|------------------------|
| 1 | 04      | 50    | AK0010200 | 23000003        | 20230101      | 1                      |
| 2 | 04      | 50    | AK0010200 | 23000005        | 20230101      | 1                      |
| 3 | 04      | 50    | AK0010200 | 23000011        | 20230102      | 1                      |
| 4 | 04      | 50    | AK0010200 | 23000013        | 20230102      | 1                      |
| 5 | 04      | 50    | AK0010200 | 23000014        | 20230102      | 1                      |

  

|   | ucr_offense_code_1 | ucr_offense_code_2 | ucr_offense_code_3 | ucr_offense_code_4 |
|---|--------------------|--------------------|--------------------|--------------------|
| 1 | 23D                | <NA>               | <NA>               | <NA>               |
| 2 | 13B                | <NA>               | <NA>               | <NA>               |
| 3 | 23C                | <NA>               | <NA>               | <NA>               |
| 4 | 13C                | <NA>               | <NA>               | <NA>               |
| 5 | 13A                | 290                | <NA>               | <NA>               |

  

|   | ucr_offense_code_5 | ucr_offense_code_6 | ucr_offense_code_7 | ucr_offense_code_8 |
|---|--------------------|--------------------|--------------------|--------------------|
| 1 | <NA>               | <NA>               | <NA>               | <NA>               |
| 2 | <NA>               | <NA>               | <NA>               | <NA>               |
| 3 | <NA>               | <NA>               | <NA>               | <NA>               |
| 4 | <NA>               | <NA>               | <NA>               | <NA>               |
| 5 | <NA>               | <NA>               | <NA>               | <NA>               |

  

|   | ucr_offense_code_9 | ucr_offense_code_10 | type_of_victim | age_of_victim |
|---|--------------------|---------------------|----------------|---------------|
| 1 | <NA>               | <NA>                | I              | 61            |
| 2 | <NA>               | <NA>                | I              | 30            |
| 3 | <NA>               | <NA>                | B              | <NA>          |
| 4 | <NA>               | <NA>                | I              | 19            |
| 5 | <NA>               | <NA>                | I              | 31            |

  

|   | sex_of_victim | race_of_victim | ethnicity_of_victim | resident_status_of_victim |
|---|---------------|----------------|---------------------|---------------------------|
| 1 | M             | W              | N                   | R                         |
| 2 | F             | W              | N                   | R                         |
| 3 | <NA>          | <NA>           | <NA>                | <NA>                      |
| 4 | F             | I              | N                   | N                         |
| 5 | M             | W              | N                   | N                         |

  

|   | circumstance_1 | circumstance_2 | additional_justifiable_homicide_circumstances |
|---|----------------|----------------|-----------------------------------------------|
| 1 | <NA>           | <NA>           | <NA>                                          |
| 2 | <NA>           | <NA>           | <NA>                                          |
| 3 | <NA>           | <NA>           | <NA>                                          |
| 4 | <NA>           | <NA>           | <NA>                                          |
| 5 | 06             | <NA>           | <NA>                                          |

  

|  | injury_1 | injury_2 | injury_3 | injury_4 | injury_5 | offender_number_to_be_related |
|--|----------|----------|----------|----------|----------|-------------------------------|
|--|----------|----------|----------|----------|----------|-------------------------------|

|                                                                            |      |      |      |      |      |      |
|----------------------------------------------------------------------------|------|------|------|------|------|------|
| 1                                                                          | <NA> | <NA> | <NA> | <NA> | <NA> | <NA> |
| 2                                                                          | M    | <NA> | <NA> | <NA> | <NA> | 01   |
| 3                                                                          | <NA> | <NA> | <NA> | <NA> | <NA> | <NA> |
| 4                                                                          | <NA> | <NA> | <NA> | <NA> | <NA> | 01   |
| 5                                                                          | N    | <NA> | <NA> | <NA> | <NA> | 01   |
| relationship_of_victim_to_offender relationship_2 relationship_3           |      |      |      |      |      |      |
| 1                                                                          |      |      | <NA> | <NA> | <NA> |      |
| 2                                                                          |      |      | BG   | <NA> | <NA> |      |
| 3                                                                          |      |      | <NA> | <NA> | <NA> |      |
| 4                                                                          |      |      | RU   | <NA> | <NA> |      |
| 5                                                                          |      |      | XR   | <NA> | <NA> |      |
| relationship_4 relationship_5 relationship_6 relationship_7 relationship_8 |      |      |      |      |      |      |
| 1                                                                          |      | <NA> | <NA> | <NA> | <NA> | <NA> |
| 2                                                                          |      | <NA> | <NA> | <NA> | <NA> | <NA> |
| 3                                                                          |      | <NA> | <NA> | <NA> | <NA> | <NA> |
| 4                                                                          |      | <NA> | <NA> | <NA> | <NA> | <NA> |
| 5                                                                          |      | <NA> | <NA> | <NA> | <NA> | <NA> |
| relationship_9 relationship_10 type_of_activity_officer_circumstance       |      |      |      |      |      |      |
| 1                                                                          |      | <NA> | <NA> |      |      | <NA> |
| 2                                                                          |      | <NA> | <NA> |      |      | <NA> |
| 3                                                                          |      | <NA> | <NA> |      |      | <NA> |
| 4                                                                          |      | <NA> | <NA> |      |      | <NA> |
| 5                                                                          |      | <NA> | <NA> |      |      | <NA> |
| assignment_type_officer ori_other_jurisdiction_officer                     |      |      |      |      |      |      |
| 1                                                                          |      | <NA> |      |      | <NA> |      |
| 2                                                                          |      | <NA> |      |      | <NA> |      |
| 3                                                                          |      | <NA> |      |      | <NA> |      |
| 4                                                                          |      | <NA> |      |      | <NA> |      |
| 5                                                                          |      | <NA> |      |      | <NA> |      |

Each row is one victim connected to one incident. `type_of_victim` distinguishes an individual person (I) from a business (B), government (G), society (S), or victim types. Many NIBRS offenses (shoplifting, vandalism, fraud, drug offenses) have a business or society as the “victim” rather than a person.

| Code | Type of victim         |
|------|------------------------|
| I    | Individual             |
| B    | Business               |
| F    | Financial Institution  |
| G    | Government             |
| R    | Religious Organization |
| S    | Society/Public         |

| Code | Type of victim          |
|------|-------------------------|
| L    | Law Enforcement Officer |
| O    | Other                   |
| U    | Unknown                 |

For individual victims, NIBRS records `age_of_victim`, `sex_of_victim` (M/F), `race_of_victim`, and `ethnicity_of_victim` as separate fields. The `ucr_offense_code_1` (through `_10`) columns record which offenses in the incident this particular victim was connected to, since a single incident can have several offenses and several victims who are not all tied to every offense.

A natural question is how victim demographics differ across crime types. We already know how to count victims by sex within each crime using `GROUP BY`. But we often want each count expressed as a *percentage of that crime's victims*, and that requires dividing each group's count by the total for its crime. With the tools we have so far, that means computing the per-crime totals in one query and joining them back. A **window function** does it in a single step.

A window function computes an aggregate across a set of related rows but, unlike `GROUP BY`, it does not collapse those rows. Instead it attaches the computed value to every row. You write an ordinary aggregate followed by an `OVER` clause that defines the “window” of rows to aggregate over. The simplest window is `OVER ()`, meaning “all rows in the result.” Here we compute each race's share of all individual victims. We use a CTE to get the per-race counts first, then a window function to divide by the grand total.

```
dbGetQuery(con, "
  WITH victim_race AS (
    SELECT race_of_victim,
           COUNT(*) AS n
    FROM   victim
    WHERE  type_of_victim = 'I'      -- individuals only
    GROUP BY race_of_victim
  )
  SELECT race_of_victim,
         n,
         ROUND(100.0 * n / SUM(n) OVER (), 1) AS pct_of_victims
  FROM   victim_race
  ORDER BY n DESC")
```

|   | race_of_victim | n | pct_of_victims |
|---|----------------|---|----------------|
| 1 | W 6071751      |   | 61.8           |
| 2 | B 2565051      |   | 26.1           |
| 3 | U 763164       |   | 7.8            |



|   |   |        |     |
|---|---|--------|-----|
| 4 | A | 310635 | 3.2 |
| 5 | I | 90000  | 0.9 |
| 6 | P | 27845  | 0.3 |
| 7 | M | 2      | 0.0 |

`SUM(n) OVER ()` adds up `n` across every row of the result and reports that single grand total alongside each row, so dividing gives each race's percentage. Without window functions we would have needed a second query (or a self-join) just to get that total. Normally you would see the aggregate function `SUM()` in the `SELECT` clause and automatically start thinking that you need a `GROUP BY` to make this work. Here, however, we do not want the query to collapse across any group. The `OVER ()` clause communicates how we want `SUM()` to aggregate, and it simply attaches that total to every row.

The real power comes from `PARTITION BY`, which splits the rows into groups and restarts the window aggregate within each group, much like `GROUP BY` but without collapsing the rows. The query below breaks down victim sex *within* each crime type. The `PARTITION BY offense_lookup.crime` makes `SUM(COUNT(*))` total only the victims of that one crime, so the percentages sum to 100 within each crime.

```
a <- dbGetQuery(con, "
  SELECT offense_lookup.crime,
         victim.sex_of_victim,
         COUNT(*) AS n,
         ROUND(100.0 * COUNT(*) /
              SUM(COUNT(*)) OVER (PARTITION BY offense_lookup.crime), 1) AS pct_within_crime
  FROM   victim
        INNER JOIN offense_lookup
              ON victim.ucr_offense_code_1 = offense_lookup.ucr_code
 WHERE  victim.type_of_victim = 'I'
        AND victim.sex_of_victim IN ('M','F')
 GROUP BY offense_lookup.crime, victim.sex_of_victim
 ORDER BY offense_lookup.crime, victim.sex_of_victim")
a
```

|   | crime              | sex_of_victim | n      |
|---|--------------------|---------------|--------|
| 1 | Aggravated Assault | F             | 328014 |
| 2 | Aggravated Assault | M             | 382838 |
| 3 | All Other Larceny  | F             | 515915 |
| 4 | All Other Larceny  | M             | 576944 |
| 5 | Arson              | F             | 9317   |
| 6 | Arson              | M             | 11954  |
| 7 | Bribery            | F             | 191    |

|    |                                            |   |        |
|----|--------------------------------------------|---|--------|
| 8  | Bribery                                    | M | 280    |
| 9  | Burglary/Breaking & Entering               | F | 244504 |
| 10 | Burglary/Breaking & Entering               | M | 301333 |
| 11 | Counterfeiting/Forgery                     | F | 30127  |
| 12 | Counterfeiting/Forgery                     | M | 32458  |
| 13 | Credit Card/Automated Teller Machine Fraud | F | 78864  |
| 14 | Credit Card/Automated Teller Machine Fraud | M | 56075  |
| 15 | Destruction/Damage/Vandalism of Property   | F | 507765 |
| 16 | Destruction/Damage/Vandalism of Property   | M | 467185 |
| 17 | Embezzlement                               | F | 3663   |
| 18 | Embezzlement                               | M | 4374   |
| 19 | Extortion/Blackmail                        | F | 5948   |
| 20 | Extortion/Blackmail                        | M | 19962  |
| 21 | False Pretenses/Swindle/Confidence Game    | F | 135564 |
| 22 | False Pretenses/Swindle/Confidence Game    | M | 116436 |
| 23 | Fondling                                   | F | 75748  |
| 24 | Fondling                                   | M | 16419  |
| 25 | Hacking/Computer Invasion                  | F | 3356   |
| 26 | Hacking/Computer Invasion                  | M | 2451   |
| 27 | Human Trafficking, Commercial Sex Acts     | F | 1567   |
| 28 | Human Trafficking, Commercial Sex Acts     | M | 276    |
| 29 | Human Trafficking, Involuntary Servitude   | F | 343    |
| 30 | Human Trafficking, Involuntary Servitude   | M | 225    |
| 31 | Identity Theft                             | F | 97653  |
| 32 | Identity Theft                             | M | 88557  |
| 33 | Impersonation                              | F | 34711  |
| 34 | Impersonation                              | M | 32725  |
| 35 | Incest                                     | F | 970    |
| 36 | Incest                                     | M | 279    |
| 37 | Intimidation                               | F | 414048 |
| 38 | Intimidation                               | M | 263271 |
| 39 | Justifiable Homicide                       | F | 28     |
| 40 | Justifiable Homicide                       | M | 663    |
| 41 | Kidnapping/Abduction                       | F | 37913  |
| 42 | Kidnapping/Abduction                       | M | 11077  |
| 43 | Motor Vehicle Theft                        | F | 339249 |
| 44 | Motor Vehicle Theft                        | M | 450229 |
| 45 | Murder & Non-negligent Manslaughter        | F | 3543   |
| 46 | Murder & Non-negligent Manslaughter        | M | 12480  |
| 47 | Negligent Manslaughter                     | F | 711    |
| 48 | Negligent Manslaughter                     | M | 1299   |
| 49 | Pocket-picking                             | F | 13923  |
| 50 | Pocket-picking                             | M | 10257  |

|    |                                             |   |         |
|----|---------------------------------------------|---|---------|
| 51 | Purse-snatching                             | F | 9466    |
| 52 | Purse-snatching                             | M | 2376    |
| 53 | Rape                                        | F | 76176   |
| 54 | Rape                                        | M | 2472    |
| 55 | Robbery                                     | F | 65540   |
| 56 | Robbery                                     | M | 132109  |
| 57 | Sexual Assault With An Object               | F | 6191    |
| 58 | Sexual Assault With An Object               | M | 1099    |
| 59 | Shoplifting                                 | F | 13439   |
| 60 | Shoplifting                                 | M | 17806   |
| 61 | Simple Assault                              | F | 1262536 |
| 62 | Simple Assault                              | M | 868924  |
| 63 | Sodomy                                      | F | 10851   |
| 64 | Sodomy                                      | M | 8416    |
| 65 | Statutory Rape                              | F | 6991    |
| 66 | Statutory Rape                              | M | 1097    |
| 67 | Stolen Property Offenses                    | F | 32459   |
| 68 | Stolen Property Offenses                    | M | 49260   |
| 69 | Theft From Building                         | F | 131325  |
| 70 | Theft From Building                         | M | 112124  |
| 71 | Theft From Coin-Operated Machine or Device  | F | 652     |
| 72 | Theft From Coin-Operated Machine or Device  | M | 1026    |
| 73 | Theft From Motor Vehicle                    | F | 398013  |
| 74 | Theft From Motor Vehicle                    | M | 514532  |
| 75 | Theft of Motor Vehicle Parts or Accessories | F | 94291   |
| 76 | Theft of Motor Vehicle Parts or Accessories | M | 138288  |
| 77 | Welfare Fraud                               | F | 3625    |
| 78 | Welfare Fraud                               | M | 1453    |
| 79 | Wire Fraud                                  | F | 20935   |
| 80 | Wire Fraud                                  | M | 16897   |
|    | pct_within_crime                            |   |         |
| 1  | 46.1                                        |   |         |
| 2  | 53.9                                        |   |         |
| 3  | 47.2                                        |   |         |
| 4  | 52.8                                        |   |         |
| 5  | 43.8                                        |   |         |
| 6  | 56.2                                        |   |         |
| 7  | 40.6                                        |   |         |
| 8  | 59.4                                        |   |         |
| 9  | 44.8                                        |   |         |
| 10 | 55.2                                        |   |         |
| 11 | 48.1                                        |   |         |
| 12 | 51.9                                        |   |         |

|    |      |
|----|------|
| 13 | 58.4 |
| 14 | 41.6 |
| 15 | 52.1 |
| 16 | 47.9 |
| 17 | 45.6 |
| 18 | 54.4 |
| 19 | 23.0 |
| 20 | 77.0 |
| 21 | 53.8 |
| 22 | 46.2 |
| 23 | 82.2 |
| 24 | 17.8 |
| 25 | 57.8 |
| 26 | 42.2 |
| 27 | 85.0 |
| 28 | 15.0 |
| 29 | 60.4 |
| 30 | 39.6 |
| 31 | 52.4 |
| 32 | 47.6 |
| 33 | 51.5 |
| 34 | 48.5 |
| 35 | 77.7 |
| 36 | 22.3 |
| 37 | 61.1 |
| 38 | 38.9 |
| 39 | 4.1  |
| 40 | 95.9 |
| 41 | 77.4 |
| 42 | 22.6 |
| 43 | 43.0 |
| 44 | 57.0 |
| 45 | 22.1 |
| 46 | 77.9 |
| 47 | 35.4 |
| 48 | 64.6 |
| 49 | 57.6 |
| 50 | 42.4 |
| 51 | 79.9 |
| 52 | 20.1 |
| 53 | 96.9 |
| 54 | 3.1  |
| 55 | 33.2 |

|    |      |
|----|------|
| 56 | 66.8 |
| 57 | 84.9 |
| 58 | 15.1 |
| 59 | 43.0 |
| 60 | 57.0 |
| 61 | 59.2 |
| 62 | 40.8 |
| 63 | 56.3 |
| 64 | 43.7 |
| 65 | 86.4 |
| 66 | 13.6 |
| 67 | 39.7 |
| 68 | 60.3 |
| 69 | 53.9 |
| 70 | 46.1 |
| 71 | 38.9 |
| 72 | 61.1 |
| 73 | 43.6 |
| 74 | 56.4 |
| 75 | 40.5 |
| 76 | 59.5 |
| 77 | 71.4 |
| 78 | 28.6 |
| 79 | 55.3 |
| 80 | 44.7 |

Notice the nesting `SUM(COUNT(*))`: the inner `COUNT(*)` is the ordinary aggregate produced by the `GROUP BY` (the number of victims of a given crime and sex), and the outer `SUM(...) OVER (PARTITION BY ...)` adds those group counts together within each crime to give the crime's total.

Standard SQL has no built in “pivot” option, but we can do that in R to get a different view on these results.

```
library(tidyr)
a |>
  pivot_wider(id_cols = crime,
              names_from = sex_of_victim,
              values_from = pct_within_crime) |>
  arrange(desc(F)) |>
  # first 6 and last 6
  slice(1:6, (n()-5):n())
```

```
# A tibble: 12 x 3
  crime                                F      M
  <chr>                             <dbl> <dbl>
1 Rape                             96.9   3.1
2 Statutory Rape                   86.4  13.6
3 Human Trafficking, Commercial Sex Acts  85    15
4 Sexual Assault With An Object     84.9  15.1
5 Fondling                         82.2  17.8
6 Purse-snatching                  79.9  20.1
7 Theft From Coin-Operated Machine or Device 38.9  61.1
8 Negligent Manslaughter            35.4  64.6
9 Robbery                          33.2  66.8
10 Extortion/Blackmail              23    77
11 Murder & Non-negligent Manslaughter 22.1  77.9
12 Justifiable Homicide             4.1  95.9
```

The results reveal stark differences in who is victimized by different crimes, with offenses such as forcible rape skewing heavily female while robbery skews male.

## 7.7 Indexes

Every time SQL evaluates a `WHERE` clause like `ucr_offense_code = '13A'`, it scans every row in the table from top to bottom looking for matches... a “full table scan.” With 14 million rows in the offense table, this can be slow. An index works like the index at the back of a textbook. Instead of reading every page to find every mention of “robbery,” you look it up in the index and jump directly to the right pages. SQLite maintains the index as a sorted structure, so finding all rows with a given `ucr_offense_code` takes milliseconds instead of seconds.

Let’s time a query before and after adding an index on `ucr_offense_code`.

```
system.time(
  dbGetQuery(con, "
    SELECT ucr_offense_code, COUNT(*) AS n
    FROM   offense
    GROUP BY ucr_offense_code
    ORDER BY n DESC")
)
```

```
user  system elapsed
1.21   0.23   1.66
```

Now let's build an index.

```
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_offense_ucr ON offense(ucr_offense_code)")
```

```
[1] 0
```

And try the query again now that it has an index handy.

```
system.time(  
  dbGetQuery(con, "  
    SELECT ucr_offense_code, COUNT(*) AS n  
    FROM   offense  
    GROUP BY ucr_offense_code  
    ORDER BY n DESC")  
)
```

| user | system | elapsed |
|------|--------|---------|
| 0.86 | 0.08   | 0.95    |

The query runs noticeably faster because SQLite can now jump directly to each offense type rather than scanning all rows. You do not change the query itself. SQLite automatically uses the index when it recognizes that it will help. Columns that appear frequently in `WHERE` clauses or `JOIN` conditions are the best candidates for indexes. Let's add a few more that will speed up the joins and filters we have been running.

```
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_offense_ori ON offense(ori)")
```

```
[1] 0
```

```
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_victim_ori  ON victim(ori, incident_number)")
```

```
[1] 0
```

```
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_arrest_ori  ON arrestee(ori, incident_number)")
```

```
[1] 0
```

Indexes do have a cost: they take up disk space and slow down `INSERT` and `UPDATE` operations slightly as the index must be kept up to date. Since we built the database once and will only query it going forward, the cost here is negligible.

Always disconnect from the database when you are done to ensure any pending writes are flushed.

```
dbDisconnect(con)
```

## 8 Exercises

1. How many Group A offenses were reported to the Philadelphia Police Department (ORI PAPEP0000)?
2. What is the most commonly reported crime in Philadelphia? Join the `offense_lookup` table to replace UCR codes with crime descriptions.
3. How many offenses are shootings? That is, the offense was aggravated assault (13A), completed (C), and the weapon involved was a firearm (codes 11, 12, 13, 14, or 15 in any of the `weapon_force` columns).
4. What is the race distribution of shooting victims? Join the `offense` table (to identify shootings) with the `victim` table. Use the victim's UCR offense codes to confirm the victim was connected to the shooting offense.
5. How many cases are recorded as justifiable homicides? Note the special circumstance codes below. Check both the `offense` table (UCR code 09C) and the `victim` table (CIRCUMSTANCE.1 codes 20 and 21).

| Code | Description                                                                           |
|------|---------------------------------------------------------------------------------------|
| 20   | Criminal killed by private citizen                                                    |
| 21   | Criminal killed by police officer                                                     |
| A    | Criminal attacked police officer and that officer killed criminal                     |
| B    | Criminal attacked fellow police officer and criminal killed by another police officer |
| C    | Criminal attacked a civilian                                                          |
| D    | Criminal attempted flight from a crime                                                |
| E    | Criminal killed in commission of a crime                                              |
| F    | Criminal resisted arrest                                                              |
| G    | Unable to determine/not enough information                                            |



6. Which agencies report the most complete data? Using the `batch_header` table, count the number of agencies by `state_abbreviation` that reported all 12 months of data (where none of the `january` through `december` columns equal “NNN”). Report the top 10 states by number of fully-reporting agencies.

## 9 Answers to the exercises

1. How many Group A offenses were reported to the Philadelphia Police Department (ORI PAPEP0000)?

```
dbGetQuery(con, "
SELECT COUNT(*) AS offense_count
FROM   offense
WHERE  ori='PAPEP0000'")
```

|   | offense_count |
|---|---------------|
| 1 | 170168        |

2. What is the most commonly reported crime in Philadelphia? Join the `offense_lookup` table to replace UCR codes with crime descriptions.

```
dbGetQuery(con, "
SELECT offense_lookup.crime,
       COUNT(*) AS offense_count
FROM   offense
       INNER JOIN offense_lookup
         ON offense.ucr_offense_code = offense_lookup.ucr_code
WHERE  offense.ori = 'PAPEP0000'
GROUP BY offense_lookup.crime
ORDER BY offense_count DESC
LIMIT 1")
```

|   | crime                                    | offense_count |
|---|------------------------------------------|---------------|
| 1 | Destruction/Damage/Vandalism of Property | 31068         |

3. How many offenses are shootings?

```
dbGetQuery(con, "
SELECT COUNT(*) AS shooting_count
FROM offense
WHERE ucr_offense_code = '13A'
      AND offense_attempted_completed = 'C'
      AND (weapon_force_1 IN ('11','12','13','14','15') OR
            weapon_force_2 IN ('11','12','13','14','15') OR
            weapon_force_3 IN ('11','12','13','14','15'))")
```

```
shooting_count
1          181371
```

4. What is the race distribution of shooting victims?

```
dbGetQuery(con, "
WITH
  shooting_incidents AS (
    SELECT DISTINCT ori, incident_number
    FROM offense
    WHERE ucr_offense_code = '13A'
          AND offense_attempted_completed = 'C'
          AND (weapon_force_1 IN ('11','12','13','14','15') OR
                weapon_force_2 IN ('11','12','13','14','15') OR
                weapon_force_3 IN ('11','12','13','14','15'))
  )
SELECT victim.race_of_victim,
       COUNT(*) AS victim_count
FROM shooting_incidents
     INNER JOIN victim
       ON shooting_incidents.ori = victim.ori
       AND shooting_incidents.incident_number = victim.incident_number
WHERE victim.ucr_offense_code_1 = '13A'
      OR victim.ucr_offense_code_2 = '13A'
GROUP BY victim.race_of_victim
ORDER BY victim_count DESC")
```

```
race_of_victim victim_count
1              B          135696
2              W          117065
3              U           10228
4              A           3059
```

|   |   |      |
|---|---|------|
| 5 | I | 2391 |
| 6 | P | 543  |

5. How many cases are recorded as justifiable homicides?

Using the offense table:

```
dbGetQuery(con, "
SELECT COUNT(*) AS justifiable_homicide_count
FROM offense
WHERE ucr_offense_code = '09C'")
```

|   |                            |
|---|----------------------------|
|   | justifiable_homicide_count |
| 1 | 671                        |

Using the victim table, joining `circumstance_lookup` to replace the codes with descriptions:

```
dbGetQuery(con, "
SELECT circumstance_lookup.circumstance,
       COUNT(*) AS victim_count
FROM victim
      INNER JOIN circumstance_lookup
      ON victim.circumstance_1 = circumstance_lookup.circumstance_code
WHERE victim.circumstance_1 IN ('20','21')
GROUP BY circumstance_lookup.circumstance")
```

|   |                                    |              |
|---|------------------------------------|--------------|
|   | circumstance                       | victim_count |
| 1 | Criminal Killed by Police Officer  | 248          |
| 2 | Criminal Killed by Private Citizen | 443          |

6. Which agencies report the most complete data?

```
dbGetQuery(con, "
SELECT state_abbreviation,
       COUNT(*) AS fully_reporting_agencies
FROM batch_header
WHERE january   != 'NNN'
      AND february != 'NNN'
      AND march   != 'NNN'
      AND april   != 'NNN'
      AND may     != 'NNN'")
```

```

AND june      != 'NNN'
AND july      != 'NNN'
AND august    != 'NNN'
AND september != 'NNN'
AND october   != 'NNN'
AND november  != 'NNN'
AND december  != 'NNN'
GROUP BY state_abbreviation
ORDER BY fully_reporting_agencies DESC
LIMIT 10")

```

|    | state_abbreviation | fully_reporting_agencies |
|----|--------------------|--------------------------|
| 1  | TX                 | 1063                     |
| 2  | IL                 | 601                      |
| 3  | CA                 | 589                      |
| 4  | MI                 | 548                      |
| 5  | OK                 | 446                      |
| 6  | OH                 | 430                      |
| 7  | KY                 | 424                      |
| 8  | MO                 | 396                      |
| 9  | MA                 | 382                      |
| 10 | MN                 | 379                      |